



Strathmore University

@iLabAfrica Centre

Applied Machine Learning

From First Principles to Production Systems

Javier Serrano
Emmanuel Lee Bundi

Master in Data Science and Analytics
DSA 8401

Nairobi, 2026

Applied Machine Learning: From First Principles to Production Systems in the African Fintech Ecosystem

Javier Serrano & Emmanuel Lee Bundi
iLabAfrica — Strathmore University, Nairobi, Kenya

Lecture notes for DSA 8401 — Applied Machine Learning,
Master in Data Science and Analytics.

© 2026 The authors. All rights reserved.
Draft edition — for enrolled students. Not for redistribution.

Typeset with L^AT_EX using the `book` class.
Figures produced with `TikZ`; code listings with `listings`.

Contact: jgarcia@strathmore.edu

Version 0.1 — July 27, 2026

Contents

Preface

vii

I	Foundations and Classical Machine Learning	1
1	Introduction to Machine Learning	3
1.1	Why This Chapter Comes First	3
1.2	What Is Machine Learning?	4
1.2.1	An operational definition	4
1.2.2	ML inverts the direction of programming	5
1.2.3	When ML earns its keep — and when it does not	5
1.2.4	The formal core: empirical risk minimization	6
1.2.5	The Bayes-optimal reference	8
1.2.6	Notation for the rest of the book	8
1.3	The Machine-Learning Landscape	8
1.3.1	The four learning paradigms	8
1.3.2	Two orthogonal operational axes	10
1.3.3	No free lunch, and what it licenses	12
1.4	Why ML Systems Fail in Production	12
1.4.1	The small green box	12
1.4.2	The notebook-to-production gap, localized	13
1.5	The End-to-End ML Lifecycle	14
1.5.1	From CRISP-DM to a monitoring-driven loop	14
1.5.2	It is a loop, not a waterfall	15
1.6	Framing the Problem and Choosing the Metric	15
1.6.1	Step 1 is where projects are won	15
1.6.2	Regression metrics: how the business pays for errors	16
1.6.3	Classification under imbalance: why accuracy lies	16
1.7	A First Look at the Data	18
1.7.1	The raw material	18
1.7.2	A feature dictionary, declared up front	19
1.7.3	What to interrogate, and why	19
1.8	Data, Splits, and Leakage	20
1.8.1	Splits: the discipline that protects your estimate of $R(f)$	20
1.8.2	Data leakage: the silent killer of fintech models	21
1.9	Baselines, Evaluation, and Generalization	23
1.9.1	Why generalization is hard: the bias–variance decomposition	23
1.9.2	Always ship a baseline first	24
1.9.3	Cross-validation: cheap insurance on the estimate	24

1.10	Presenting and Deciding	25
1.10.1	The decision memo	25
1.10.2	Champion, challenger, and shadow mode	25
1.11	Fairness, Explainability, and Regulation	26
1.11.1	Making “fair” precise enough to test	26
1.11.2	Explainability as a deployment requirement	27
1.12	From Model to System: MLOps and the Agentic Outlook	27
1.12.1	The MLOps spine of this course	27
1.12.2	Where LLMs and agents fit — and where they do not	28
1.13	Chapter Summary	29
	Exercises	31
	Key Terms	32
	Further Reading	32
2	Data Preparation and Feature Engineering	3
2.1	Introduction: From Raw Data to Model-Ready Data	3
2.1.1	The Six Dimensions of Data Quality	4
2.1.2	The Unit of Prediction	5
2.2	Missing Data	5
2.2.1	Rubin’s Taxonomy of Missingness	5
2.2.2	Diagnosis Before Treatment	6
2.2.3	Imputation I: Simple Statistics with Indicators	7
2.2.4	Imputation II: Model-Based Methods	7
2.2.5	A Decision Framework	8
2.3	Outliers and Duplicates	8
2.3.1	Detecting Outliers	9
2.3.2	Treating Outliers — and When Not To	9
2.3.3	Duplicates and Entity Resolution	10
2.4	Scaling and Transformations	10
2.4.1	Why Scale, and With What	10
2.4.2	Taming Skew: Log and Power Transforms	11
2.5	Encoding Categorical Variables	12
2.5.1	Low Cardinality: One-Hot and Ordinal	12
2.5.2	High Cardinality I: Frequency Encoding and the Hashing Trick	12
2.5.3	High Cardinality II: Target Encoding	13
2.6	Feature Engineering	13
2.6.1	Engineered versus Learned Features	14
2.6.2	Domain Features for Mobile-Money Credit Scoring	14
2.6.3	Datetime Features and Cyclical Encoding	14
2.6.4	Interactions, Binning, and Crosses	15
2.6.5	What Makes a Feature Good?	16
2.7	Data Leakage	16
2.7.1	Definition and Why It Is Dangerous	16
2.7.2	Six Common Causes	17
2.7.3	Temporal and Group-Aware Splits	17
2.7.4	A Detection Protocol	18
2.7.5	The Structural Antidote: Pipelines	18
2.8	From Pipelines to Feature Platforms	19
	Assignment 2 (graded, 20 marks)	20
	Exercises	21
	Key Terms	21

Further Reading	21
3 Classical Machine Learning and Rigorous Evaluation	37
3.1 Introduction	37
3.2 Core Theory	37
3.3 Implementation	37
 II Deep Learning and Generative AI	 39
 III MLOps and Production Systems	 41
 A Capstone Project	 29
A.1 Project Brief	29
A.2 Suggested Themes (Localised)	29
A.3 Week 14: Deploy & Test (“War Room”)	30
A.4 Week 15: Final Presentations	30
A.5 Grading Rubric	30
 Appendices	 31
 B Solutions to Selected Exercises	 31
Chapter 1	31
B.1 Chapter 2: Data Preparation and Feature Engineering	36

Preface

This book collects the lecture notes of **DSA 8401 — Applied Machine Learning**, taught in the Master in Data Science and Analytics at iLabAfrica, Strathmore University. It is written for graduate students who already program and know basic statistics, and who want to go beyond training models in a notebook: the goal throughout is *production-grade* machine learning — data pipelines, rigorous evaluation, deployment, and monitoring.

Three commitments shape every chapter:

1. **An end-to-end view.** Models are treated as components of systems. MLOps (Docker, FastAPI, MLflow, CI/CD) is not an afterthought but the destination of the whole arc.
2. **African context as pedagogy.** Worked examples are anchored in the African technology and financial ecosystem: thin-file mobile-money credit scoring (the running thread of the course), trade-corridor underwriting, clinical pathway modelling, and streaming fraud detection. This is a deliberate instructional choice, not decoration.
3. **Modern architectures.** Transformers, LLMs, RAG pipelines, and an outlook on agentic systems are integrated where they matter for applied work.

How this book maps to the course

Each chapter corresponds to one teaching week (three hours: a 1.5-hour lecture plus a 1.5-hour practical lab). The two MLOps weeks are consolidated into a single chapter, and the capstone project brief lives in Appendix A.

Chapter	Topic	Course week
1	End-to-End Machine Learning	1
2	Data Preparation & Feature Engineering	2
3	Classical ML & Rigorous Evaluation	3
4	Ensemble Methods & Hyperparameter Tuning	4
5	Unsupervised Learning & Dim. Reduction	5
6	Deep Learning Fundamentals	6
7	Computer Vision with CNNs	7
8	Sequence Models & Time Series	9
9	Transformers, LLMs & RAG	10
10	MLOps: Serving, Tracking & CI/CD	11–12
11	Monitoring, Drift & System Design	13
A	Capstone Project	14–15

Prerequisites and companion material

We assume the Data Mining unit, working Python, and comfort with linear algebra and probability at the level of a first graduate course. The primary textbook companion is [Géron \(2022\)](#); [Bishop and Bishop \(2024\)](#) and [Huyen \(2022\)](#) support the deep-learning and MLOps parts respectively. Each chapter closes with a lab section that mirrors the Jupyter notebook used in the practical session.

Javier Serrano & Emmanuel Lee Bundi
Nairobi, 2026

Part I

Foundations and Classical Machine Learning

Introduction to Machine Learning

Week 1

Learning Objectives

By the end of this chapter — and the accompanying lab — you should be able to:

- State a precise, operational definition of machine learning and decide, for a concrete business problem, whether ML is the right tool at all.
- Formalize supervised learning as *empirical risk minimization* and explain why the gap between empirical and expected risk is the central engineering problem of applied ML.
- Situate any given problem within the ML taxonomy (supervised, unsupervised, self-supervised, reinforcement) and along the two operational axes (batch vs. online; instance- vs. model-based).
- Enumerate the dominant failure modes of ML systems in production — technical debt, training/serving skew, drift, and leakage — and their amplifiers in African fintech deployments.
- Walk the eight-step end-to-end lifecycle for a real problem: frame it, choose a defensible metric, design honest data splits, build a leakage-free baseline pipeline, and articulate the path to deployment and monitoring.
- State testable group-fairness criteria, explain why they conflict and why dropping the protected attribute does not produce fairness, and describe how explainability requirements constrain model choice in regulated credit.
- Explain where LLMs and agentic patterns genuinely add value in a modern applied-ML architecture — and where classical models remain the correct core.

1.1 Why This Chapter Comes First

Most machine-learning curricula open with an algorithm: linear regression, perhaps, or k -nearest neighbours. This book opens instead with a *system*. The reason is empirical: the overwhelming majority of ML projects that fail in industry do not fail because the wrong algorithm was chosen. They fail because the problem was framed against the wrong objective, because the evaluation was silently optimistic, because the data pipeline that produced the training set could not be reproduced at serving time, or because nobody was watching when the world drifted away from the snapshot the model was trained on (Huyen, 2022).

This chapter is therefore the conceptual map for the whole unit. Its goal is to establish a shared, *production-grade* mental model of what an ML system — not merely a model — is, before we descend into the machinery of individual algorithms in later weeks. Everything is anchored in a single running thread that will accompany us from this page to the capstone project: a **thin-file credit-scoring problem on mobile-money data**, of the kind that a Nairobi, Lagos, or Accra fintech lender confronts daily. By the end of the unit you should be able to answer, with code and with argument, the question that this chapter poses: *what does it actually take to ship a model that keeps working?*

Case Study: Thin-File Credit Scoring on Mobile Money

A digital lender wants to extend working-capital credit to *thin-file* micro-merchants — traders, boda-boda operators, market-stall owners — who have little or no formal credit history with a credit reference bureau. What they do have is a rich behavioural exhaust: months of mobile-money transactions (deposits, withdrawals, merchant payments, peer transfers, airtime purchases), stored as timestamped event logs. Research has shown that such behavioural traces carry genuine signal about repayment, and thin-file lending on mobile rails is now one of the highest-impact applications of ML on the continent.

The lender's question sounds simple: *given an applicant's transaction history, what is the probability that a loan of amount a over term τ will default?* Turning that question into a deployed, monitored, regulated, and fair decision system is the running exercise of this entire book. Every chapter adds one ring of the lifecycle to this same problem: this week we frame it; Week 2 engineers its features without leakage; Weeks 3–4 model and evaluate it rigorously; Weeks 11–13 serve, track, and monitor it; and the capstone asks you to walk the full loop again, solo, on a theme of your choice.

1.2 What Is Machine Learning?

1.2.1 An operational definition

The definition we adopt is Tom Mitchell's, because it is *operational*: it forces you to name the three quantities that any ML project must specify before a single line of modelling code is written.

Definition 1.1 (Learning, after Mitchell 1997). A computer program is said to *learn* from experience E with respect to some task T and performance measure P if its performance at T , as measured by P , improves with experience E .

The definition looks innocuous until you try to instantiate it for a real problem. Consider fraud detection on a mobile-money rail:

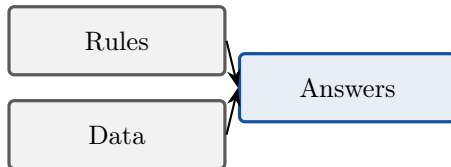
- **Task T** : flag a transaction as fraudulent at the moment it is initiated, before settlement.
- **Experience E** : eighteen months of labelled transaction logs — labels arriving days or weeks late, from chargebacks, customer complaints, and manual investigations.
- **Performance P** : cost-weighted recall at a *fixed alert budget*, because the fraud-operations team can investigate only a few hundred alerts per day.

Notice how much engineering reality is already hiding inside E and P : the labels are delayed and noisy; the metric is not accuracy but a constrained, cost-sensitive quantity dictated by operations. Machine learning, on this view, is the discipline of turning E into a function that we can *deploy*, *evaluate*, and *maintain* — three verbs that will recur throughout this book.

1.2.2 ML inverts the direction of programming

Figure 1.1 contrasts the two paradigms. In classical programming, a human encodes *rules*; the program applies them to *data* to produce *answers*. In machine learning the arrow is inverted: we supply data *and* answers (labels), and the learning algorithm *induces* the rules — the model — that map one to the other. This inversion is what makes ML economically attractive in domains where the rules are unknown, unstable, or too numerous to enumerate; it is also what makes ML systems operationally fragile, because the induced rules are only as good as the data they were induced from, and the data keeps changing.

Classical programming



Machine learning

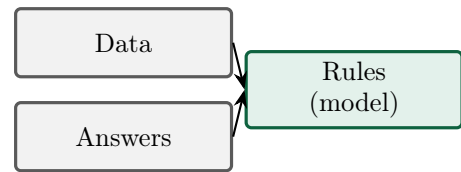


Figure 1.1: Machine learning inverts classical programming: instead of coding the rules by hand, we induce them from examples. The induced rules — the model — are the small green box; [Section 1.4](#) argues that the *system* around that box is where most of the engineering lives.

1.2.3 When ML earns its keep — and when it does not

The most expensive mistake in applied ML is committed before any data is loaded: choosing ML when a rule would do, or a rule when only ML can cope. [Table 1.1](#) summarizes the decision.

Use ML when ...	Prefer rules / heuristics when ...
There is no tractable closed-form rule — fraud patterns shift weekly, and a hand-written rulebook cannot keep up.	A short, auditable rule already solves the problem to the required standard.
The environment <i>drifts</i> : a static rulebook rots as behaviour, adversaries, and regulation change.	Data is scarce, noisy, or unlabelled, and more cannot realistically be obtained.
The problem is high-dimensional or strongly non-linear — hundreds of interacting behavioural features.	Errors are catastrophic <i>and</i> every decision must be fully explainable (some regulated credit decisions).
You want to <i>extract structure</i> from data: segmentation, anomaly discovery, embeddings.	The total cost of owning a maintained ML system exceeds the value it adds over the rule.

Table 1.1: The build-or-not decision. The right-hand column is not a list of ML’s failures but of its *costs*: an ML system is a liability under permanent maintenance ([Section 1.5](#)), and that liability must be paid for by measurable lift over the simplest alternative.

A useful engineering discipline follows: **start with the simplest thing that could possibly work, then justify every added gram of ML complexity**. For our credit-scoring thread, the incumbent is a manual scorecard maintained by credit officers. Any ML system we build must beat that scorecard — not on accuracy, but on the business quantity it exists to optimize (expected loss at a given approval rate, [Section 1.6](#)) — by enough margin to pay for its own data pipelines, serving infrastructure, and monitoring. This “lift over baseline” framing recurs in [Section 1.9](#) and is graded explicitly in the capstone rubric.

Remark 1.1. The dichotomy in Table 1.1 is not permanent. A common production pattern — and a healthy one — is *rules first, ML second*: launch with an auditable heuristic to start collecting labelled outcomes, then train against those outcomes once volume justifies it. The rule then survives as the baseline the model must beat, and often as a guard-rail wrapped around the model’s output.

1.2.4 The formal core: empirical risk minimization

We now give the mathematical skeleton on which all of supervised learning — Weeks 1 through 10 — hangs. Suppose examples (x, y) are drawn independently and identically distributed (i.i.d.) from an unknown joint distribution $\mathcal{D}$ over $\mathcal{X} \times \mathcal{Y}$, where $x \in \mathcal{X}$ is a feature vector (an applicant’s engineered transaction summary) and $y \in \mathcal{Y}$ is the target (default within the loan term, $y \in \{0, 1\}$). We choose a *hypothesis class* $\mathcal{H}$ — a family of candidate functions $f : \mathcal{X} \rightarrow \mathcal{Y}$ (or to scores/probabilities) — and a *loss function* $L(y, f(x)) \geq 0$ quantifying the cost of predicting $f(x)$ when the truth is y . What we would *like* to minimize is the **expected risk** (also called the true risk or generalization error):

$$R(f) = \mathbb{E}_{(x,y) \sim \mathcal{D}} [L(y, f(x))]. \quad (1.1)$$

The distribution $\mathcal{D}$ is unknown — if we knew it, there would be nothing to learn — so we cannot compute Eq. (1.1). What we have instead is a finite training sample $S = \{(x_i, y_i)\}_{i=1}^n \sim \mathcal{D}^n$, on which we can compute the **empirical risk** and minimize it, typically with a capacity-controlling *regularizer* $\Omega(f)$ weighted by $\lambda \geq 0$:

$$\hat{f} = \arg \min_{f \in \mathcal{H}} \underbrace{\frac{1}{n} \sum_{i=1}^n L(y_i, f(x_i))}_{\hat{R}(f)} + \lambda \Omega(f). \quad (1.2)$$

Three remarks on the ingredients of Eq. (1.2):

- **The loss L encodes the business.** Squared error $L(y, \hat{y}) = (y - \hat{y})^2$ suits regression with symmetric, thin-tailed error costs; the logarithmic loss

$$L(y, \hat{p}) = -y \log \hat{p} - (1 - y) \log(1 - \hat{p})$$

suits probabilistic classification and rewards *calibration*; and a *cost-sensitive* loss that weights false negatives (missed defaults) differently from false positives (declined good customers) is what our lender actually cares about (Section 1.6.2).

- **The regularizer Ω controls capacity.** Ridge ($\Omega(f) = \|w\|_2^2$) and lasso ($\Omega(f) = \|w\|_1$) penalties for linear models, depth and leaf constraints for trees, dropout and weight decay for neural networks — all are instances of the same idea: shrink the effective size of $\mathcal{H}$ so that fitting the sample does not degenerate into memorizing it. The multiplier λ trades data fit against complexity and is the prototypical *hyperparameter* (tuned properly in Week 4).
- **Everything downstream exists because $\hat{R}(\hat{f}) \neq R(\hat{f})$.** Splits, cross-validation, leakage discipline, drift monitoring — the entire apparatus of Sections 1.8–1.9 and of the MLOps weeks — exists to *estimate* the gap between empirical and expected risk honestly, and then to *keep that gap closed* after deployment. This sentence is the thesis of the course: *the whole engineering problem of applied ML is closing the gap between $\hat{R}$ and R , and keeping it closed over time.*

How large can the gap be? For a finite hypothesis class, a classical concentration argument (Hoeffding's inequality plus a union bound) gives, with probability at least $1 - \delta$ over the draw of the sample,

$$\sup_{f \in \mathcal{H}} |R(f) - \hat{R}(f)| \leq \sqrt{\frac{\ln|\mathcal{H}| + \ln(2/\delta)}{2n}}. \quad (1.3)$$

The bound is loose and rarely computed in practice, but its *shape* is worth memorizing: the generalization gap grows with the (log) size of the hypothesis class and shrinks only as $1/\sqrt{n}$ with the sample size. Rich model families demand either lots of data or strong regularization; and no amount of cleverness on a fixed sample substitutes for more, and more representative, data. Crucially, [Eq. \(1.3\)](#) assumes the i.i.d. sampling of [Eq. \(1.1\)](#) — an assumption that production deployment *systematically violates* as the serving distribution drifts away from the training snapshot. That violation is not a footnote; it is the subject of [Section 1.4](#) and of Week 13.

Example 1.1 (Computing the empirical risk in code). [Listing 1.1](#) makes [Eq. \(1.2\)](#) concrete: it computes the empirical log-loss of a probabilistic classifier by hand, verifies it against scikit-learn, and shows the effect of the regularization weight λ (exposed as $C = 1/\lambda$ in `LogisticRegression`).

```

1 import numpy as np
2 from sklearn.linear_model import LogisticRegression
3 from sklearn.metrics import log_loss
4
5 rng = np.random.default_rng(8401)
6 n, d = 5_000, 12                                # applicants, engineered features
7 X = rng.normal(size=(n, d))
8 true_w = rng.normal(size=d)
9 p_true = 1 / (1 + np.exp(-X @ true_w))
10 y = rng.binomial(1, p_true)                       # default indicator
11
12 # Fit a regularized logistic model: C = 1/lambda
13 clf = LogisticRegression(C=1.0, max_iter=1_000).fit(X, y)
14 p_hat = clf.predict_proba(X)[: , 1]
15
16 # Empirical risk under log-loss, computed from the definition
17 eps = 1e-15
18 p_clipped = np.clip(p_hat, eps, 1 - eps)
19 R_hat = -np.mean(y * np.log(p_clipped)
20                + (1 - y) * np.log(1 - p_clipped))
21
22 assert np.isclose(R_hat, log_loss(y, p_hat))
23 print(f"Empirical risk (log-loss): {R_hat:.4f}")
24
25 # Stronger regularization (larger lambda, smaller C) raises training
26 # loss but can lower *expected* risk -- the bias-variance trade-off
27 # of Section 1.8 in action.
28 for C in [100.0, 1.0, 0.01]:
29     m = LogisticRegression(C=C, max_iter=1_000).fit(X, y)
30     print(f"C={C:>6}: train log-loss ="
31           f" {log_loss(y, m.predict_proba(X)[: , 1]):.4f}")

```

Listing 1.1: Empirical risk (log-loss) by hand and via scikit-learn; the regularization strength is the knob λ of [Eq. \(1.2\)](#).

Note what [Listing 1.1](#) does *not* do: it evaluates the model on the same data it was trained on, so the printed numbers estimate $\hat{R}(\hat{f})$, not $R(\hat{f})$. Taking training-set performance as a proxy for deployment performance is the original sin of applied ML; [Section 1.8](#) builds the discipline that prevents it.

1.2.5 The Bayes-optimal reference

It is worth asking what the best achievable value of Eq. (1.1) is, if we place no restriction on f at all. The minimizer over *all* measurable functions is called the **Bayes-optimal predictor**, and its risk the **Bayes risk** $R^* = \inf_f R(f)$. For two canonical losses the optimum has a clean closed form:

- **Squared loss (regression).** The pointwise optimum is the conditional expectation,

$$f^*(x) = \mathbb{E}[y \mid x],$$

and the Bayes risk equals the average conditional variance of y given x — exactly the irreducible-noise term σ^2 that reappears in the bias–variance decomposition of Section 1.9.1.

- **0–1 loss (classification).** The optimum is the maximum a posteriori rule,

$$f^*(x) = \arg \max_{c \in \mathcal{Y}} P(y = c \mid x),$$

whose risk is $R^* = \mathbb{E}_x[1 - \max_c P(y = c \mid x)]$ — strictly positive whenever the classes genuinely overlap in feature space, as defaulters and repayers always do.

The Bayes predictor is uncomputable (it requires the very distribution $\mathcal{D}$ we do not know), but it plays two practical roles. First, it is the *ceiling*: reported metrics should be discussed relative to a plausible ceiling, not to perfection — a point we weaponize against unrealistic stakeholder expectations in Section 1.9.1. Second, it cleanly separates the two sources of error in any deployed model: the *approximation–estimation* gap between our $\hat{f}$ and f^* (which better models, features, and data can shrink) and the Bayes risk itself (which nothing can). Decomposing

$$R(\hat{f}) - R^* = \underbrace{\left(\inf_{f \in \mathcal{H}} R(f) - R^* \right)}_{\text{approximation error}} + \underbrace{\left(R(\hat{f}) - \inf_{f \in \mathcal{H}} R(f) \right)}_{\text{estimation error}}$$

shows the fundamental tension: enlarging $\mathcal{H}$ shrinks the first term and inflates the second (recall Eq. (1.3)); regularization navigates between them. This is the bias–variance trade-off in its decision-theoretic clothing.

1.2.6 Notation for the rest of the book

Table 1.2 fixes the symbols used throughout the following chapters; deviations are declared locally.

1.3 The Machine-Learning Landscape

Before diving deeper into the supervised setting, we survey the territory. The purpose of a taxonomy is not classification for its own sake: naming the learning paradigm correctly tells you what *signal* you have, what *algorithms* are admissible, and what *evaluation* is honest.

1.3.1 The four learning paradigms

Supervised learning.

The training data contains input–output pairs (x_i, y_i) ; the goal is to predict y for unseen x . When $\mathcal{Y}$ is discrete we speak of *classification* (will this applicant default: yes/no); when continuous, of *regression* (what repayment fraction should we expect). Supervised learning is where the ERM machinery of Section 1.2.4 applies most directly, and it is where most deployable fintech value concentrates — which is why Weeks 3–4 (Chapter 3 and ??) live here.

Symbol	Meaning
$x \in \mathcal{X}, y \in \mathcal{Y}$	feature vector and target; $\mathcal{X} \subseteq \mathbb{R}^d$ unless stated otherwise
$\mathcal{D}; S = \{(x_i, y_i)\}_{i=1}^n$	unknown data distribution; training sample of size n drawn i.i.d. from $\mathcal{D}$
$\mathcal{H}; f, \hat{f}, f^*$	hypothesis class; a hypothesis, the learned model, the Bayes-optimal predictor
$L(y, \hat{y})$	loss function; ℓ_2 , log-loss, or cost-sensitive as declared
$R(f); \hat{R}(f); R^*$	expected risk Eq. (1.1) ; empirical risk; Bayes risk
$\Omega(f), \lambda$	regularizer and its strength (scikit-learn's $C = 1/\lambda$)
$\hat{p} = \hat{P}(y=1 x)$	estimated positive-class (default) probability
t, t^*	decision threshold; cost-optimal threshold Eq. (1.6)
c_{FN}, c_{FP}	unit costs of false negatives and false positives
$P(x), P(y x)$	input marginal and target conditional; drift is defined on these (Definition 1.2)

Table 1.2: Standing notation for the book.

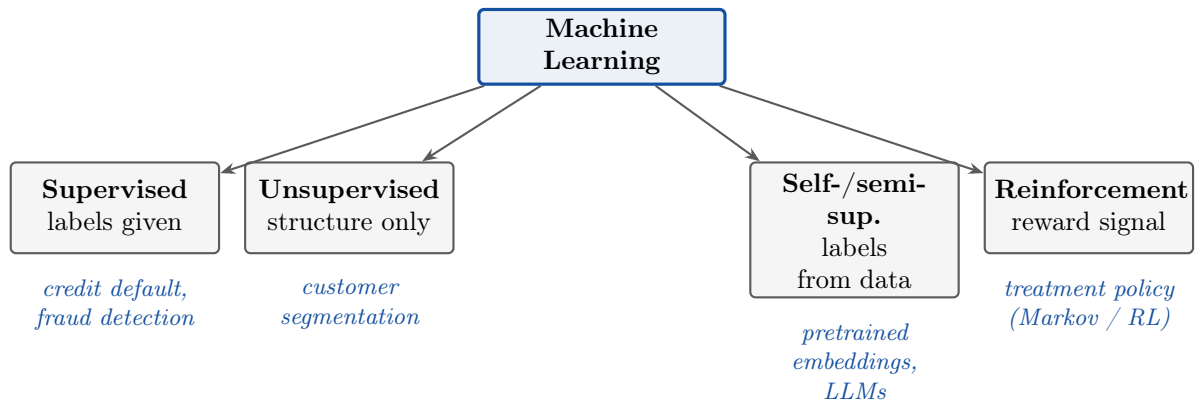


Figure 1.2: The four learning paradigms, each annotated with its natural application in our applied domain. The boundaries blur in modern systems: an LLM is self-supervised pretraining plus reinforcement learning from human feedback (RLHF); an agentic underwriter wraps a supervised scorer inside an RL-style decision loop.

Unsupervised learning.

Only inputs x_i are given; the goal is to discover *structure*: clusters of similar merchants, low-dimensional representations of transaction behaviour, anomalous KYC profiles that deserve human review. There is no ground truth to score against, which makes honest evaluation genuinely harder (Week 5).

Self- and semi-supervised learning.

Labels are manufactured from the data itself (predict a masked token, a held-out transaction, the next event in a sequence) or a small labelled set is combined with a large unlabelled one. This is the paradigm behind pretrained transaction embeddings and, at scale, behind large language models — which are, at their core, self-supervised next-token predictors refined with reinforcement learning (Week 10).

Reinforcement learning.

An *agent* interacts with an environment over time, observing states, taking actions, and receiving rewards; the goal is a *policy* maximizing cumulative reward. Sequential clinical treatment planning over Markov models of patient pathways is the canonical health-informatics example, and RL framing also fits dynamic credit-limit management. In this unit RL appears as an elective lens and a capstone option rather than a core week.

Table 1.3 maps the four paradigms onto our domain and onto the course schedule.

Paradigm	Signal	African applied example	Course week
Supervised	(x, y) pairs	Thin-file credit default classification on mobile-money logs	Wk 3–4
Unsupervised	x only	Merchant / borrower segmentation; KYC anomaly clusters	Wk 5
Self-supervised	pretext labels from x	Transaction embeddings; LLMs for document underwriting	Wk 10
Reinforcement	reward r_t	Sequential treatment policy via Markov / RL on clinical pathways	elective / capstone

Table 1.3: The four paradigms mapped to the applied domain of this course. We spend most of the unit in the supervised regime, then layer generative and agentic patterns on top of it.

1.3.2 Two orthogonal operational axes

Independently of the learning paradigm, two architectural choices shape every deployment. They sound academic; they are anything but — they set your latency budget, retraining cadence, and infrastructure bill, which is exactly what the MLOps weeks formalize.

Batch versus online learning

A **batch** (offline) system is retrained periodically — nightly, weekly, monthly — on a snapshot of accumulated data, validated, and then promoted to serving. It is simple, reproducible, and auditable, and it is the correct default for credit scoring, where decisions tolerate seconds of latency and regulators expect versioned, reviewable models. Its weakness is *lag*: between retrains, the model is blind to change.

An **online** (incremental) system updates its parameters continuously, per event or per mini-batch, as data streams in. Streaming fraud detection on payment rails is the motivating case:

attack patterns can shift within hours, faster than any batch cadence. The price is operational: online learners are harder to validate, vulnerable to poisoned or corrupted input streams, and require the ability to roll back to a known-good state when (not if) a bad update lands.

[Listing 1.2](#) contrasts the two modes on a simulated transaction stream via the `partial_fit` interface of scikit-learn, which performs stochastic gradient descent one mini-batch at a time.

```

1 import numpy as np
2 from sklearn.linear_model import SGDClassifier
3 from sklearn.preprocessing import StandardScaler
4
5 rng = np.random.default_rng(8401)
6
7 def transaction_batch(t, size=512):
8     """Simulate a mini-batch whose fraud pattern drifts with time t."""
9     X = rng.normal(size=(size, 8))
10    drift = 0.15 * t # concept drift: weights move
11    w = np.array([1.5, -2.0, 0.8, 0, 0, 0.5, -1.0, 0.3]) + drift
12    p = 1 / (1 + np.exp(-(X @ w - 3.0))) # rare positive class
13    y = rng.binomial(1, p)
14    return X, y
15
16 scaler = StandardScaler()
17 online = SGDClassifier(loss="log_loss", alpha=1e-4,
18                       random_state=8401)
19
20 for t in range(20): # 20 arriving mini-batches
21     X_t, y_t = transaction_batch(t)
22     X_t = (scaler.partial_fit(X_t).transform(X_t))
23     online.partial_fit(X_t, y_t, classes=[0, 1])
24     if t % 5 == 0:
25         X_next, y_next = transaction_batch(t + 1) # prequential eval:
26         acc = online.score(scaler.transform(X_next), y_next)
27         print(f"batch {t:2d}: next-batch accuracy = {acc:.3f}")

```

Listing 1.2: Batch versus online learning on a transaction stream. The online model ingests mini-batches via `partial_fit`; a real deployment would add validation gates and rollback around each update.

Note the evaluation style in [Listing 1.2](#): each model is scored on the *next* batch before training on it (“prequential” or test-then-train evaluation). This respects time order — the same discipline, in streaming form, as the temporal splits of [Section 1.8](#). [Table 1.4](#) condenses the trade-off.

Instance-based versus model-based learning

An **instance-based** learner memorizes the training set and predicts by comparison: k -nearest neighbours scores a new applicant by finding the k most similar historical applicants and voting. Training is trivially cheap; *serving* is expensive, because every prediction touches the stored data, and the whole training set becomes a runtime dependency of the production system.

A **model-based** learner compresses the training set into a fixed set of parameters — the weights of a logistic regression, the splits of a tree ensemble — and discards the data at serving time. Serving is fast and cheap; the model artifact is small, versionable, and shippable inside a container. For production scoring under latency and cost budgets, model-based methods are the default, and this economic asymmetry — not any statistical superiority — is the main reason you will rarely meet k -NN behind a payments API.

	Batch (offline)	Online (incremental)
Update cadence	periodic retrain on snapshots	per event / mini-batch
Freshness vs. drift	lags between retrains	tracks fast drift
Validation	full offline evaluation before promotion	prequential; gates and rollback around every update
Reproducibility & audit	strong: versioned data + model	harder: state depends on stream history
Attack surface	poisoning must survive a re-train review	poisoned stream corrupts the live model
Natural fit here	credit scoring, limit setting	streaming fraud on payment rails

Table 1.4: Batch versus online learning as an operational decision. Most production systems are batch with a deliberately chosen retraining cadence; going online is a response to drift speed, paid for in validation and audit complexity.

1.3.3 No free lunch, and what it licenses

A last piece of orientation before we leave the landscape. The *no-free-lunch* theorems state, informally, that averaged over all possible data-generating problems, every learning algorithm performs identically: no model family is universally superior. The theorem is sometimes quoted nihilistically; its practical reading is the opposite, and it underwrites three habits this course drills:

- **Priors are unavoidable, so choose them consciously.** Any algorithm that generalizes does so because its inductive bias matches the structure of *this* problem — smoothness, sparsity, feature interactions, temporal locality. “Which model is best?” is ill-posed; “which inductive bias fits mobile-money default risk?” is an engineering question with an empirical answer.
- **Shortlist and compare; never anoint.** Because no family wins a priori, step 4 of the lifecycle trains *several* diverse families quickly and lets honest validation (Section 1.9.3) adjudicate. In practice, on the tabular, moderate-sized data of our domain, gradient-boosted trees very often win — but that is an empirical regularity about a *class* of problems, not a theorem, and it is re-verified on every project.
- **Benchmarks are domain-relative.** Leaderboard supremacy on one distribution transfers to yours only insofar as the distributions share structure — one more reason the localized datasets of this course matter pedagogically.

1.4 Why ML Systems Fail in Production

1.4.1 The small green box

The single most cited systems paper in machine learning makes an uncomfortable observation: in a mature production deployment, the ML code — the part this book’s middle chapters teach — is a *tiny fraction* of the total system. The rest is data collection and verification, feature extraction, serving infrastructure, monitoring, configuration, and resource management (Fig. 1.3).

Worse, ML does not merely inherit ordinary software’s technical debt; it adds *new* categories of its own:

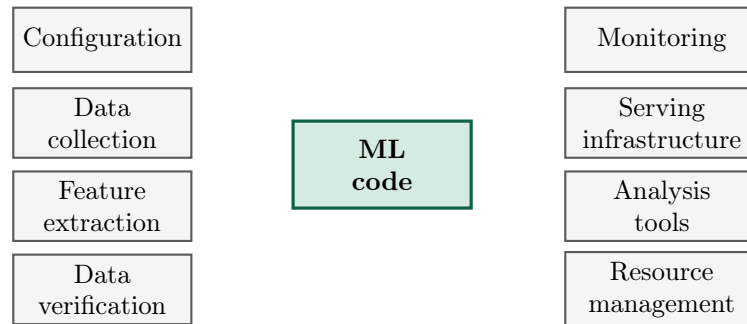


Figure 1.3: Schematic after: the green core is the model; everything else is the system. A great notebook is not a system.

Entanglement (CACE).

“Changing Anything Changes Everything.” Features interact through the learned weights: improving one input feature can silently degrade the model’s use of every other. No feature change is ever local.

Hidden feedback loops.

The model’s own decisions shape the future training data. A credit model that declines a segment never observes that segment’s repayment behaviour again — so its beliefs about the segment can never be corrected by data. (This *selective labels* problem is endemic to lending and returns in Week 3.)

Training/serving skew.

The feature computed offline for training and the “same” feature computed online at serving time diverge — different code paths, different aggregation windows, different null handling. The model then sees, in production, inputs from a distribution it never trained on.

Undeclared consumers and silent data dependencies.

Downstream systems quietly consume a model’s outputs, and upstream data providers quietly change schemas or semantics. Either can break the system without a single test failing.

1.4.2 The notebook-to-production gap, localized

The generic failure modes above are amplified by the realities of deploying in African fintech, which is precisely why this course insists on treating them in Week 1 rather than as an afterthought:

- **Thin files.** Applicants have sparse, short histories; per-user signal is weak, sample sizes for rare outcomes are small, and the temptation to over-engineer features (and leak) is correspondingly large.
- **Alternative data is noisy and shifting.** Airtime top-ups, mobile-money velocity, agent-network behaviour: rich signal, but its semantics move with tariff changes, promotions, competitor entry, and regulation — structural drift, not gentle statistical drift.
- **Connectivity and latency at the edge.** Scoring may need to complete within an in-app interaction over a congested mobile network; models may need to be small, cached, or partially on-device.
- **Regulatory and fairness pressure.** Automated credit decisions face growing explainability and non-discrimination requirements; “the model said so” is not a defence, and disparate performance across regions or genders is a launch blocker, not a footnote.

Production Pitfall

The model is rarely why projects fail; the *system* around it is. Concretely, the four killers you must design against from day one are: (i) **training/serving skew** — eliminate it by sharing one feature pipeline between training and serving (Section 1.8, Week 2); (ii) **data and concept drift** — $\mathcal{D}_{\text{train}} \neq \mathcal{D}_{\text{serve}}$ over time; detect it with monitoring, respond with retraining (Week 13); (iii) **leakage** — beautiful offline metrics, production collapse (Section 1.8.2); and (iv) **irreproducibility** — “works on my machine” is a failure state; pin environments, version data and code, seed randomness (today’s lab).

Let us make drift, the second killer, precise, because the terms are often conflated. Write the joint distribution as $P(x, y) = P(y | x) P(x)$.

Definition 1.2 (Data drift vs. concept drift). **Data drift** (covariate shift) is a change in the input distribution $P(x)$ while $P(y | x)$ stays fixed: a new marketing campaign brings in younger applicants, but the relationship between behaviour and repayment is unchanged. **Concept drift** is a change in the conditional $P(y | x)$ itself: after a macroeconomic shock, the *same* transaction profile now carries a different default risk. Label-prior shift, a change in $P(y)$, is a third, often intermediate, case.

The distinction matters operationally: data drift can sometimes be absorbed by reweighting or by retraining on recent data, whereas concept drift invalidates the learned mapping itself and may demand re-framing, new features, or a new model. Monitoring systems (Week 13) are designed to detect both — input-distribution monitors for the former, delayed-label performance monitors for the latter.

1.5 The End-to-End ML Lifecycle

1.5.1 From CRISP-DM to a monitoring-driven loop

Structured processes for data projects long predate the current ML wave — CRISP-DM (Cross-Industry Standard Process for Data Mining, 1999) already prescribed business understanding, data understanding, preparation, modelling, evaluation, and deployment. Géron’s widely used eight-step checklist (Géron, 2022) refines this for ML practice. The modern correction, which this book adopts as doctrine, is that the sequence is not a pipeline that terminates at deployment: it is a **loop**, and *monitoring is the trigger that starts the next turn* (Fig. 1.4).

1. **Frame the problem** — business objective, ML task type, metric, baseline, constraints (Section 1.6).
2. **Get and explore the data** — acquisition, profiling, exploratory analysis; understand missingness, imbalance, time structure.
3. **Prepare the data** — cleaning, feature engineering, leakage-safe transformation pipelines (Week 2).
4. **Shortlist and train models** — several families, quickly, against the baseline (Weeks 3–7).
5. **Fine-tune and select** — hyperparameter search, cross-validated comparison, error analysis (Week 4).
6. **Present and decide** — communicate lift over baseline in business terms; secure the go/no-go.

7. **Deploy** — package, containerize, serve behind an API, integrate with CI/CD (Weeks 11–12).
8. **Monitor and maintain** — watch inputs, outputs, and delayed outcomes; retrain or roll back on drift (Week 13).

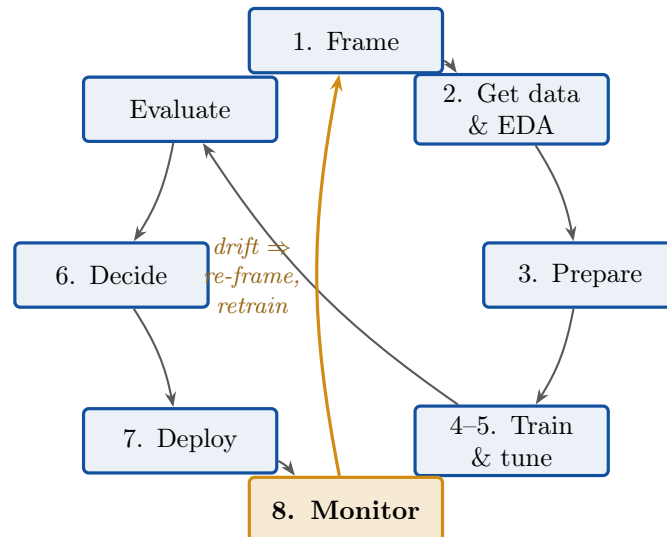


Figure 1.4: The end-to-end lifecycle as a loop. The amber edge is the one that distinguishes a production discipline from a class project: monitoring feeds re-framing and retraining, so “done” never arrives.

1.5.2 It is a loop, not a waterfall

Four consequences of the loop view are worth internalizing now, because they invert the instincts most of us bring from coursework:

1. **Iterate fast on the early steps.** Each turn of the loop is cheap to start and expensive to finish. A day spent sharpening the framing or discovering a data pathology saves weeks of modelling against the wrong target.
2. **Eighty percent of the work lives in steps 1–3.** Framing and data dominate the effort budget of real projects; model selection, the part that dominates textbooks, is a minority activity. This book’s week allocation mirrors that reality.
3. **A deployed model is a liability under maintenance,** not a finished artifact. It decays by default ([Definition 1.2](#)); drift detection, retraining, and rollback are first-class design requirements, not operational afterthoughts.
4. **You will walk the loop twice.** This unit walks the entire loop once across Weeks 1–13 on the shared credit-scoring thread, then has you walk it again — solo, end to end — in the capstone ([Chapter A](#)).

1.6 Framing the Problem and Choosing the Metric

1.6.1 Step 1 is where projects are won

We now execute step 1 of the lifecycle on the running case. A lender wants to extend working-capital credit to thin-file micro-merchants using mobile-money history. Framing means writing down, before any modelling, four things:

1. **The business objective.** Approve more good borrowers without raising default losses past a threshold — equivalently, maximize approved volume subject to a portfolio expected-loss constraint. Note that this is *not* “predict default accurately”: the model is an instrument, not the objective.
2. **The ML problem type.** Supervised *binary classification*, estimating $P(\text{default} \mid x)$ for applicant features x , followed by a *cost-sensitive decision threshold* that converts the probability into approve/decline/refer. Separating the *prediction* problem from the *decision* problem is a recurring design principle: the same probability model can serve many decision policies as risk appetite changes.
3. **The current baseline.** A manual scorecard maintained by credit officers. Any ML system must beat it on *expected loss at matched approval rate* — not on accuracy, AUC, or any other proxy — by enough to fund its own maintenance (Section 1.2.3).
4. **The constraints.** Explainability for the regulator (reason codes for declines); latency for in-app decisions (sub-second scoring); fairness across regions and genders (audited disparate performance); and data-protection compliance for behavioural data.

If you cannot write down the objective, the metric, and the baseline, you are not ready to touch a model. In this course, every lab report and the capstone brief begin with exactly this framing block, and it is graded.

1.6.2 Regression metrics: how the business pays for errors

When the target is continuous — predicting the repayment fraction, or the sustainable credit limit — the two workhorse metrics are the root-mean-square error and the mean absolute error over m evaluation points:

$$\text{RMSE} = \sqrt{\frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i)^2}, \quad \text{MAE} = \frac{1}{m} \sum_{i=1}^m |\hat{y}_i - y_i|. \quad (1.4)$$

They are the empirical risks of the squared and absolute losses, respectively, and they encode different economics. RMSE, being quadratic, punishes large errors disproportionately: it is the right choice when the cost of error grows faster than linearly — fat-tailed loss distributions, where one badly mis-sized credit limit can wipe out the margin on a hundred good ones. MAE weights all errors linearly and is robust to outliers: the right choice when errors cost roughly in proportion to their size. The selection rule is not statistical taste but a business question: *how does the organization actually pay for a mistake of size ε ?* (A related fact worth remembering: the constant prediction minimizing squared error is the conditional *mean*; the one minimizing absolute error is the conditional *median*. Your loss choice silently decides which summary of the target you are learning.)

1.6.3 Classification under imbalance: why accuracy lies

Default events are rare — perhaps 5% of loans in a healthy portfolio. On such data, a degenerate model that predicts “never default” for everyone achieves 95% accuracy and is completely useless: it approves every bad loan. Accuracy is the wrong lens whenever classes are imbalanced *and* error costs are asymmetric, which in fintech is essentially always.

The honest vocabulary starts from the confusion matrix. Writing TP, FP, TN, FN for true/false positives/negatives at a given decision threshold:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad \text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \quad F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}.$$

Precision answers “of the applicants we flagged as risky, how many truly were?”; recall answers “of the truly risky, how many did we catch?”. [Table 1.5](#) fixes the layout and the derived rates.

		Predicted	
		risky ($\hat{y} = 1$)	safe ($\hat{y} = 0$)
Actual	default ($y = 1$)	TP	FN (missed default, cost c_{FN})
	repaid ($y = 0$)	FP (lost customer, cost c_{FP})	TN

Table 1.5: The confusion matrix at a fixed threshold, annotated with the business cost of each error type for the lending case. Recall = $TP/(TP+FN)$ is also called the true-positive rate (TPR); the false-positive rate is $FPR = FP/(FP+TN)$.

Sweeping the threshold traces two standard curves ([Fig. 1.5](#)): the ROC curve (TPR against FPR) and the precision–recall curve. Their areas, ROC–AUC and PR–AUC, summarize ranking quality across all thresholds. Under heavy imbalance, PR–AUC is the more discriminating summary, because ROC–AUC can look flattering merely by ranking the abundant negatives correctly: with 95% negatives, a model can post an ROC–AUC of 0.90 while its precision at useful recall levels remains poor — a fact the PR curve exposes immediately, since its baseline is the prevalence, not the 0.5 diagonal.

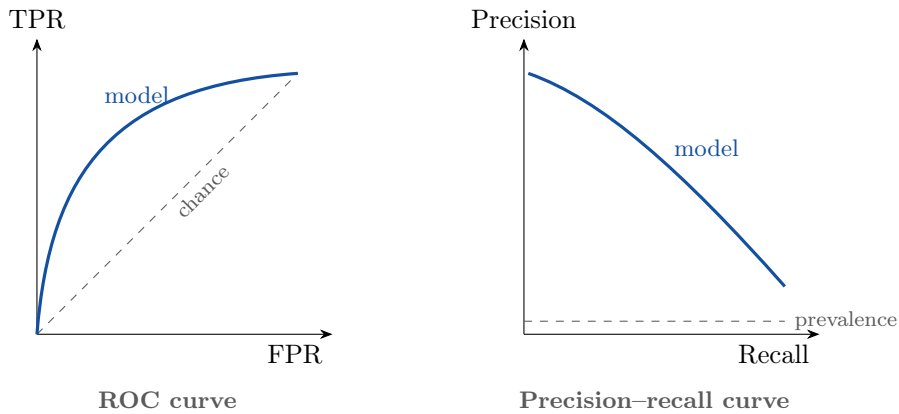


Figure 1.5: ROC and precision–recall views of the same imbalanced classifier. The chance baselines differ: the ROC diagonal is fixed, while the PR baseline equals the prevalence (here $\approx 5\%$) — which is why PR–AUC is the harsher, more informative summary under imbalance.

But ranking quality is still a proxy. What the lender ultimately optimizes is **expected cost**. If a missed default (false negative) costs c_{FN} — the written-off principal — and a wrongly declined good customer (false positive) costs c_{FP} — the foregone margin and the churned relationship — then the operating point should minimize

$$\mathbb{E}[C] = c_{FN} FN + c_{FP} FP, \quad (1.5)$$

with, typically, $c_{FN} \gg c_{FP}$ in lending. Given a *calibrated* probability estimate $\hat{p} = P(\text{default} \mid x)$, decision theory gives the optimal threshold in closed form: decline when expected default loss exceeds expected margin, i.e. when $\hat{p} c_{FN} > (1 - \hat{p}) c_{FP}$, equivalently

$$\hat{p} > t^* = \frac{c_{FP}}{c_{FP} + c_{FN}}. \quad (1.6)$$

[Equation \(1.6\)](#) is why *calibration* — the property that among applicants scored $\hat{p} = 0.2$, roughly 20% actually default — matters so much in regulated scoring, and why Week 3 treats

it as a first-class evaluation axis alongside discrimination. Listing 1.3 computes the empirical cost curve and compares the empirically best threshold with t^* .

```

1 import numpy as np
2 from sklearn.datasets import make_classification
3 from sklearn.linear_model import LogisticRegression
4 from sklearn.model_selection import train_test_split
5 from sklearn.metrics import confusion_matrix
6
7 C_FN, C_FP = 50_000, 4_000          # KES: missed default vs lost margin
8
9 X, y = make_classification(n_samples=20_000, n_features=15,
10                           weights=[0.95, 0.05],    # 5% default rate
11                           random_state=8401)
12 X_tr, X_te, y_tr, y_te = train_test_split(
13     X, y, test_size=0.25, stratify=y, random_state=8401)
14
15 p_te = (LogisticRegression(max_iter=1_000)
16         .fit(X_tr, y_tr).predict_proba(X_te)[: , 1])
17
18 thresholds = np.linspace(0.01, 0.99, 197)
19 costs = []
20 for t in thresholds:
21     tn, fp, fn, tp = confusion_matrix(y_te, p_te >= t).ravel()
22     costs.append(C_FN * fn + C_FP * fp)
23
24 best = thresholds[int(np.argmin(costs))]
25 t_star = C_FP / (C_FP + C_FN)    # optimal iff p is calibrated
26 print(f"Empirical best threshold: {best:.3f}")
27 print(f"Theoretical t* = {t_star:.3f} (valid under calibration)")
28 print(f"Cost at 0.5: {costs[np.searchsorted(thresholds, 0.5)]:>12,}")
29 print(f"Cost at best: {min(costs):>12,}")

```

Listing 1.3: Cost-sensitive threshold selection: sweep the threshold, compute the expected cost of Eq. (1.5), and compare with the closed-form optimum of Eq. (1.6).

Run it: the naive threshold of 0.5 is typically several times costlier than the cost-aware one. Nothing about the *model* changed — only the decision layer. This is the cheapest performance improvement in applied ML, and it is routinely left on the table.

Remark 1.2 (Alert budgets). Sometimes the binding constraint is not a cost ratio but *capacity*: a fraud-operations team can review, say, 300 alerts per day. The honest metric is then recall (or cost captured) *at the fixed alert budget* — precision@ k in ranking terms. Metrics must inherit the operational constraint, not idealize it away.

1.7 A First Look at the Data

Framing (step 1) done, we execute step 2 of the lifecycle on the running case: acquire the data and understand it *before* preparing or modelling it. Exploratory data analysis (EDA) is not a ritual scatter-plot session; it is a structured interrogation whose findings directly reshape the framing, the features, and the split design.

1.7.1 The raw material

The lender’s warehouse exposes three tables. The `transactions` event log is the primary asset: one row per mobile-money event (deposit, withdrawal, peer transfer, merchant payment, air-time purchase, loan disbursement, repayment), with a timestamp, amount, counterparty type, channel, and agent identifier. The `applications` table holds one row per credit application:

applicant identifier, timestamp, requested amount and term, and self-declared business sector. The `loans` table holds originated loans and their outcomes — including the field that, after careful definition, becomes our label.

Even the *label* requires framing decisions that EDA must inform. “Default” is not a column; it is a definition: days-past-due beyond a threshold (30? 60? 90?), measured at a horizon (end of term? term plus grace?). Each choice changes the base rate, the label delay (how long after origination the outcome is knowable — which bounds how fresh any training set can be), and the economics of Eq. (1.5). In this course we adopt *60 days past due within term-plus-30*, a common micro-lending convention, and we record the definition in the project README as versioned metadata: when the definition changes, every historical metric silently changes meaning with it.

1.7.2 A feature dictionary, declared up front

Week 2 engineers features properly; today we fix the vocabulary. Every feature is declared with its *as-of semantics* — the rule stating which raw events may contribute, relative to the application timestamp t_0 . Table 1.6 shows the starter dictionary used in the labs. The discipline of writing the as-of column now is what makes the leakage audit of Section 1.8.2 mechanical later.

Feature	Definition	Type	As-of
<code>txn_count_90d</code>	number of transactions in $[t_0-90d, t_0)$	int	$< t_0$
<code>inflow_median_90d</code>	median credit amount, same window	float	$< t_0$
<code>balance_vol_90d</code>	std. dev. of end-of-day balance, same window	float	$< t_0$
<code>agent_diversity_90d</code>	distinct agents transacted with, same window	int	$< t_0$
<code>airtime_spend_30d</code>	airtime purchases in $[t_0-30d, t_0)$	float	$< t_0$
<code>region</code>	registered operating region	cat	static
<code>business_sector</code>	self-declared sector at application	cat	t_0
<code>wallet_tier</code>	KYC tier of the wallet at application	cat	t_0

Table 1.6: Starter feature dictionary for the running case. The *as-of* column is a leakage contract: every value must be computable strictly from information available at the application timestamp t_0 .

1.7.3 What to interrogate, and why

Four questions dominate first-pass EDA on this kind of data, each with a downstream consequence:

1. **Missingness — how much, and why?** A missing `balance_vol_90d` may mean a data-pipeline gap (missing at random-ish) or a genuinely new wallet with no history (missing *not* at random — the missingness itself is signal). The two demand different treatment in Week 2: imputation for the former, explicit “no-history” indicators for the latter. Thin-file portfolios are, by definition, heavy in the second kind.
2. **Imbalance — what is the base rate, per segment?** A 5% global default rate can hide a 2% rate in one region and 11% in another. The global rate calibrates the metric discussion of Section 1.6.3; the per-segment rates foreshadow the fairness analysis of Section 1.11 and dictate stratification keys for the splits.

3. **Time structure — is the data stationary?** Plot volumes, base rates, and key feature distributions *by calendar month*. Promotions, tariff changes, agent-network expansion, and macro-shocks all leave visible level shifts. Any non-stationarity found here is an argument for the temporal split of [Section 1.8](#) and a preview of the drift monitoring of Week 13.
4. **Duplicates and entities — who repeats?** Count applications per applicant and loans per merchant. Repeat entities are the group-leakage hazard of [Section 1.8.2](#) and must drive the choice of a group-aware splitter.

[Listing 1.4](#) is the profiling skeleton the lab uses — deliberately plain: EDA code should be boring, fast, and re-runnable, because it will be re-run every time the data refreshes.

```

1 import pandas as pd
2
3 apps = pd.read_parquet("data/applications.parquet")
4 loans = pd.read_parquet("data/loans.parquet")
5 df = loans.merge(apps, on="application_id", validate="1:1")
6
7 # 1. Missingness: how much, and is it 'no history' or 'pipeline gap'?
8 miss = df.isna().mean().sort_values(ascending=False)
9 print(miss[miss > 0].to_string(float_format="{:.1%}".format))
10 no_history = df["txn_count_90d"].isna() & (df["wallet_age_days"] < 90)
11 print(f"missing-because-new-wallet: {no_history.mean():.1%}")
12
13 # 2. Imbalance: global and per-segment base rates
14 print(f"global default rate: {df['default_within_term'].mean():.2%}")
15 print(df.groupby("region")["default_within_term"]
16       .agg(["mean", "count"]).sort_values("mean"))
17
18 # 3. Time structure: monthly volumes and base rates
19 monthly = (df.set_index("application_date")
20            .resample("MS")["default_within_term"]
21            .agg(rate="mean", volume="size"))
22 print(monthly.tail(12))          # look for level shifts, not noise
23
24 # 4. Entities: who repeats? (drives group-aware splitting)
25 per_merchant = df.groupby("merchant_id").size()
26 print(f"merchants with >1 loan: {(per_merchant > 1).mean():.1%}, "
27       f"max loans per merchant: {per_merchant.max()}")

```

Listing 1.4: First-pass profiling of the running dataset: missingness, per-segment base rates, temporal stability, and entity repetition — each mapped to a downstream design decision.

Remark 1.3 (EDA is hypothesis generation, not hypothesis confirmation). Findings from EDA feed features and split design; they must *not* feed model selection against the test period. Looking at test-period data during EDA — even “just to check the distributions” — is a mild form of the data snooping that [Section 1.8](#) forbids. Profile the training window; seal the rest.

1.8 Data, Splits, and Leakage

1.8.1 Splits: the discipline that protects your estimate of $R(f)$

Since $R(f)$ in [Eq. \(1.1\)](#) cannot be computed, all evaluation rests on estimating it from held-out data. The integrity of that estimate is the most fragile asset in an ML project, and the split discipline exists to protect it:

- **Train / validation / test.** Fit on the training set; compare models and tune hyper-parameters on the validation set; touch the test set *once*, at the very end, to report the

final, unbiased estimate. Every additional peek at the test set leaks information from it into your modelling choices and inflates the reported metric — *data-snooping bias*. A test set consulted weekly is a validation set with a misleading name.

- **Stratify on the rare class.** With a 5% default rate, a careless random split can leave a fold with too few positives to evaluate on. Stratified splitting preserves the class ratio in every partition.
- **Split temporally for fintech.** Transactions, loans, and fraud all live in time. Train on the past, validate on the more recent past, test on the most recent period (Fig. 1.6). A random shuffle across time lets the model “see the future” — e.g. learning from June behaviour to predict March outcomes — and produces estimates that no deployed system can honour, because deployment always predicts forward.
- **Split by group when entities repeat.** If the same merchant contributes several loans, they must all land on the same side of the split; otherwise the model partially memorizes merchants rather than learning risk patterns (group leakage, below).

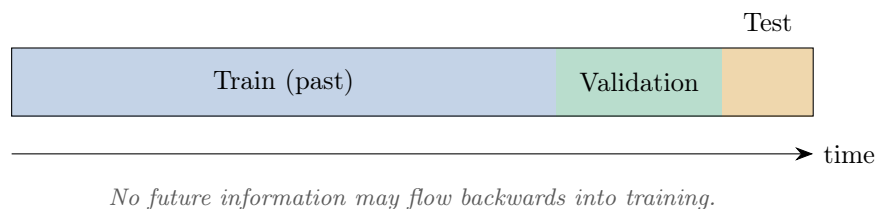


Figure 1.6: Temporal splitting for time-structured data. The test period is the most recent one, mimicking deployment, where the model always predicts forward in time.

1.8.2 Data leakage: the silent killer of fintech models

Definition 1.3 (Leakage). *Leakage* is the use, at training time, of information that will not be (honestly) available at prediction time. Its signature is a model with beautiful offline metrics that collapses in production.

Leakage is the most common and the most expensive silent failure in applied ML, and thin-file fintech data — heavy on temporal aggregates and repeated entities — is unusually fertile ground for it. Four species dominate:

Target leakage.

A feature is a proxy for, or a consequence of, the label. Classic example: a “loan written off” flag, or the number of collection calls, used to predict default. The feature exists *because* the outcome happened; at application time it will always be empty.

Temporal leakage.

A feature aggregates over a window that includes the future relative to the prediction moment — “average balance over the loan term” as a feature for a decision made at origination. Every windowed feature must be computable strictly from data timestamped *before* the decision.

Train/test contamination.

Preprocessing statistics are computed on the full dataset before splitting: a scaler fitted on all rows, an imputer whose means include test rows, target encodings computed globally. Each transfers information from test to train. The cure is structural, not vigilance-based: encapsulate every fitted transformation inside a **Pipeline** that is fit *only* on training folds (Listing 1.5).

Group leakage.

The same underlying entity (merchant, device, household) appears on both sides of the split, letting the model recognize entities instead of generalizing across them. Use group-aware splitters keyed on the entity identifier.

```

1 import pandas as pd
2 from sklearn.compose import ColumnTransformer
3 from sklearn.impute import SimpleImputer
4 from sklearn.linear_model import LogisticRegression
5 from sklearn.pipeline import Pipeline
6 from sklearn.preprocessing import OneHotEncoder, StandardScaler
7 from sklearn.metrics import average_precision_score
8
9 df = pd.read_parquet("data/loans.parquet")    # one row per loan
10 df = df.sort_values("application_date")
11
12 # --- 1. TEMPORAL split BEFORE any fitting -----
13 cutoff_valid = "2025-07-01"
14 cutoff_test  = "2025-11-01"
15 train = df[df.application_date < cutoff_valid]
16 valid = df[(df.application_date >= cutoff_valid)
17            & (df.application_date < cutoff_test)]
18 test  = df[df.application_date >= cutoff_test]    # sealed until the end
19
20 num_cols = ["txn_count_90d", "inflow_median_90d", "balance_vol_90d",
21            "agent_diversity_90d", "airtime_spend_30d"]
22 cat_cols = ["region", "business_sector", "wallet_tier"]
23 TARGET   = "default_within_term"
24
25 # --- 2. All fitted steps live INSIDE the pipeline -----
26 prep = ColumnTransformer([
27     ("num", Pipeline([("impute", SimpleImputer(strategy="median")),
28                     ("scale", StandardScaler())]), num_cols),
29     ("cat", OneHotEncoder(handle_unknown="ignore"), cat_cols),
30 ])
31 model = Pipeline([("prep", prep),
32                  ("clf", LogisticRegression(max_iter=2_000,
33                                             class_weight="balanced"))])
34
35 model.fit(train[num_cols + cat_cols], train[TARGET])
36 p_valid = model.predict_proba(valid[num_cols + cat_cols])[:, 1]
37 print("Validation PR-AUC:",
38       round(average_precision_score(valid[TARGET], p_valid), 4))
39 # The test block is evaluated ONCE, after model selection is frozen.

```

Listing 1.5: A leakage-safe skeleton: temporal split first, then a Pipeline whose every fitted step (imputation, scaling, encoding, model) learns from training data only. This same object is later serialized and served, eliminating training/serving skew by construction.

Production Pitfall

If a result looks too good, **suspect leakage first** — before celebrating, before presenting, and long before deploying. A validation PR-AUC that jumps implausibly after adding one feature is a leak detector firing, not a breakthrough. Practical audit questions for every feature: *At the exact moment of prediction, would this value exist? Computed from which data, timestamped when? Could it exist only because the outcome already happened?* Week 2 turns this audit into a systematic feature-review checklist, and a strong baseline with no leakage beats a fancy model with leakage — every single time.

1.9 Baselines, Evaluation, and Generalization

1.9.1 Why generalization is hard: the bias–variance decomposition

We can now say precisely why closing the $\hat{R}$ – R gap is non-trivial. Fix a query point x and suppose the truth is $y = g(x) + \varepsilon$ with noise $\mathbb{E}[\varepsilon] = 0$, $\text{Var}(\varepsilon) = \sigma^2$. Let $\hat{f}$ be the model our learning procedure produces from a random training sample S ; because S is random, so is $\hat{f}$. Write $\bar{f}(x) = \mathbb{E}_S[\hat{f}(x)]$ for the average prediction over training samples. Then the expected squared error at x — over both the sampling of S and the noise — decomposes exactly:

$$\begin{aligned} \mathbb{E}[(y - \hat{f}(x))^2] &= \mathbb{E}[(g(x) + \varepsilon - \hat{f}(x))^2] \\ &= \mathbb{E}[(g(x) - \hat{f}(x))^2] + \sigma^2 && (\varepsilon \perp \hat{f}, \mathbb{E}[\varepsilon] = 0) \\ &= \mathbb{E}[(g(x) - \bar{f}(x) + \bar{f}(x) - \hat{f}(x))^2] + \sigma^2 \\ &= \underbrace{(g(x) - \bar{f}(x))^2}_{\text{Bias}^2} + \underbrace{\mathbb{E}[(\bar{f}(x) - \hat{f}(x))^2]}_{\text{Variance}} + \underbrace{\sigma^2}_{\text{irreducible noise}}, \end{aligned} \quad (1.7)$$

where the cross-term vanishes because $\mathbb{E}_S[\hat{f}(x) - \bar{f}(x)] = 0$. Each term has an engineering meaning:

- **High bias (underfitting):** the model family is too rigid to represent g ; both training *and* test error are high. Remedies: richer features, a more flexible family, less regularization.
- **High variance (overfitting):** the model is flexible enough to fit the noise of the particular sample it saw; training error is low, test error high, and retraining on a fresh sample would yield a visibly different model. Remedies: more data, stronger regularization $\lambda \Omega(f)$, a simpler family, ensembling (Week 4 — averaging attacks precisely the variance term).
- **Irreducible noise σ^2 :** the floor set by information genuinely absent from x — for default prediction, the illness, theft, or market shock that no transaction history foretells. No model beats the floor; *chasing it is how you overfit*, and recognizing it protects you from stakeholders who demand near-perfect scores that no honest model can deliver.

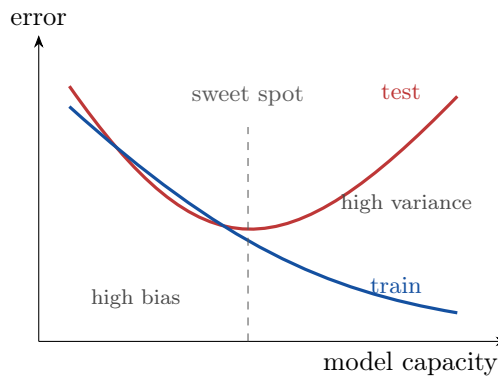


Figure 1.7: The classical capacity picture implied by Eq. (1.7): training error falls monotonically with capacity while test error is U-shaped. Regularization, more data, and simpler hypothesis classes move you along the trade-off toward the sweet spot.

Remark 1.4. Deep networks complicate this picture without invalidating it: very large, heavily over-parameterized models can exhibit “double descent”, in which test error falls a second time beyond the interpolation threshold; we return to this in Week 6. For the tabular, moderate-data regime of most fintech scoring, Fig. 1.7 remains the operative mental model.

1.9.2 Always ship a baseline first

A *baseline* is the simplest defensible predictor for the task, built and evaluated before any serious modelling:

1. **The degenerate baseline** — majority class, global mean, or the incumbent business rule — sets the floor and calibrates every subsequent number. If the scorecard already achieves a given expected loss, that, not zero, is the number to beat.
2. **The simple-model baseline** — a regularized logistic regression on honest features is often 90% of the achievable value, trains in seconds, and is explainable for free. Everything fancier must justify its marginal complexity against this reference.
3. **Report lift over baseline, never raw scores in isolation.** “PR-AUC 0.41” is uninterpretable on its own; “PR-AUC 0.41 against a prevalence of 0.05 and a scorecard baseline of 0.28” is a result.

1.9.3 Cross-validation: cheap insurance on the estimate

A single validation split yields a noisy estimate of $R(f)$, particularly with rare positives. k -fold cross-validation partitions the training data into k folds, trains k times leaving one fold out for validation each time, and averages:

$$\hat{R}_{CV} = \frac{1}{k} \sum_{j=1}^k \hat{R}_{\text{val}}^{(j)}, \quad (1.8)$$

reducing the variance of the estimate at the price of k trainings. Two adaptations are mandatory in our domain: **stratified** CV, so each fold preserves the default rate; and **time-series** CV (expanding- window or rolling-origin), so each fold trains strictly on the past and validates on the future — ordinary shuffled k -fold on temporal data is temporal leakage in disguise (Section 1.8.2). Hyperparameters are tuned on the CV folds; the test set stays sealed throughout. Listing 1.6 assembles the full week-one discipline: degenerate baseline, simple-model baseline, honest CV, and lift reporting.

```

1 import numpy as np
2 from sklearn.datasets import make_classification
3 from sklearn.dummy import DummyClassifier
4 from sklearn.linear_model import LogisticRegression
5 from sklearn.model_selection import (StratifiedKFold, TimeSeriesSplit,
6                                     cross_val_score)
7 from sklearn.pipeline import Pipeline
8 from sklearn.preprocessing import StandardScaler
9
10 X, y = make_classification(n_samples=12_000, n_features=20,
11                           n_informative=8, weights=[0.95, 0.05],
12                           random_state=8401)
13
14 # 1. Degenerate baseline: the floor every model must clear
15 floor = DummyClassifier(strategy="prior")
16
17 # 2. Simple-model baseline: scaler + regularized logistic, as ONE object
18 logit = Pipeline([("scale", StandardScaler()),
19                  ("clf", LogisticRegression(C=1.0, max_iter=2_000,
20                                           class_weight="balanced"))])
21
22 # 3. Honest estimate: stratified 5-fold CV on PR-AUC
23 cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=8401)
24 for name, est in [("majority-class floor", floor),

```

```

25         ("logistic baseline ", logit)]:
26     scores = cross_val_score(est, X, y, cv=cv,
27                             scoring="average_precision")
28     print(f"{name}: PR-AUC = {scores.mean():.3f} "
29           f" (+/- {2 * scores.std():.3f})")
30
31 # 4. For genuinely temporal data, swap in expanding-window CV:
32 #     cv = TimeSeriesSplit(n_splits=5) # rows must be time-ordered

```

Listing 1.6: The week-one evaluation discipline in one script: a majority-class floor, a leakage-safe logistic baseline, stratified cross-validation, and lift over baseline as the headline number.

1.10 Presenting and Deciding

Step 6 of the lifecycle is the one this book can least automate and the one that most often decides whether steps 7–8 ever happen: presenting the candidate system to the people who own the risk, and securing an explicit go/no-go. Graduate courses tend to treat communication as garnish; in industry it is a gating artifact, and we treat it as one — the labs, and especially the capstone presentation, are graded against the structure below.

1.10.1 The decision memo

A model that cannot be summarized in two pages for a credit-risk committee is not ready to be deployed. The memo structure this course uses:

1. **Framing recap (five lines).** Objective, ML task, metric, baseline, constraints — the block of [Section 1.6](#), verbatim. If this has drifted since kickoff, that is the first finding to report.
2. **Headline result, in business units.** Not “PR–AUC improved from 0.28 to 0.41” but “at the current approval rate, expected annual write-offs fall by KES X; alternatively, approvals can rise by Y% at unchanged expected loss.” The translation runs through the cost model of [Eq. \(1.5\)](#) and the chosen operating threshold.
3. **Evidence of honesty.** The split design ([Fig. 1.6](#)), the leakage audit outcome ([Section 1.8.2](#)), cross-validated uncertainty ([Eq. \(1.8\)](#) — report the spread, not just the mean), and per-segment performance ([Section 1.11](#)).
4. **Error analysis.** Where the model is weakest — which segments, which feature regimes, which time periods — and what that implies for referral rules and monitoring thresholds.
5. **Risks and reversibility.** What could invalidate the result (drift scenarios, upstream data changes), how quickly it would be detected (Week 13’s monitors), and the rollback plan.
6. **The ask.** A specific decision: deploy to shadow mode, deploy behind a champion/challenger split, or iterate — with the criteria that would trigger the next decision.

1.10.2 Champion, challenger, and shadow mode

Committees rarely (and should rarely) approve a hard cut-over. Two lower-risk promotion patterns dominate, and both reappear as infrastructure in Weeks 11–13:

- **Shadow mode.** The new model scores every live application but its output is logged, not acted on. After a label-maturation period, its *would-have-been* decisions are compared

with the incumbent’s on identical traffic — the cleanest possible like-for-like evidence, at the cost of waiting out the label delay.

- **Champion/challenger.** Live traffic is split: the incumbent (champion) keeps most of it; the challenger takes a small, monitored share. This produces true outcome data for the challenger — partially escaping the selective-labels trap of [Section 1.4](#) — while capping the blast radius of a bad model.

Both patterns presuppose the monitoring and versioning machinery of the MLOps weeks; the decision memo should state which pattern is proposed and why. The deeper lesson generalizes: *deployment is an experiment design problem*, and the statistics of [Section 1.9.3](#) apply to it just as they applied offline.

1.11 Fairness, Explainability, and Regulation

Framing ([Section 1.6](#)) listed fairness and explainability among the hard constraints of the running case. They deserve more than a bullet, because in African fintech they are simultaneously an ethical obligation, a regulatory requirement, and — less obviously — a *technical consequence* of the failure modes already discussed: selective labels ([Section 1.4](#)) and segment-heterogeneous data ([Section 1.7](#)) are precisely the mechanisms by which an innocently trained model becomes an unfair one.

1.11.1 Making “fair” precise enough to test

Let A denote a protected attribute (gender, region), $\hat{y}$ the model’s decision, and y the outcome. Two of the most used group-fairness criteria are:

- **Demographic parity:** the approval rate is independent of the group,

$$P(\hat{y} = 1 \mid A = a) = P(\hat{y} = 1 \mid A = a') \quad \forall a, a'.$$

- **Equalized odds:** error rates are equal across groups conditional on the true outcome,

$$P(\hat{y} = 1 \mid y = c, A = a) = P(\hat{y} = 1 \mid y = c, A = a') \quad \forall a, a', c \in \{0, 1\},$$

i.e. equal TPR and equal FPR per group in the language of [Table 1.5](#).

Three facts every practitioner must know. First, these criteria *conflict*: except in degenerate cases (equal base rates across groups, or a perfect classifier), demographic parity, equalized odds, and per-group calibration cannot all hold simultaneously — fairness is a *choice* among defensible definitions, to be made explicitly with the business and the regulator, not discovered in a library default. Second, *removing the protected attribute does not help*: region and gender are redundantly encoded in transaction patterns, agent networks, and sectors (proxy discrimination), so “fairness through unawareness” fails quietly while removing the very column needed to *audit* the model. Third, the selective-labels loop makes the audit data itself biased: we observe repayment only for approved applicants, so historical approvals shape what “the data says” about each group.

Operationally, this course’s standard is modest and enforceable: every model evaluation in the labs and the capstone reports the headline metric *and* per-segment recall, FPR, and calibration, by region and gender, with material gaps discussed — the same reporting discipline as lift-over-baseline, extended one axis.

1.11.2 Explainability as a deployment requirement

Regulated credit decisions require *reason codes*: the applicant-facing statement of the main factors behind a decline. This constrains the architecture, not just the paperwork. Two viable patterns dominate practice: (i) an inherently interpretable core — scorecard-style logistic regression on monotone features — whose coefficients *are* the explanation; or (ii) a stronger model (gradient-boosted trees, Week 4) paired with post-hoc attribution (SHAP values, Week 4) that is validated for stability before it is allowed to generate customer-facing codes. What is *not* viable is choosing the model first and hoping explainability can be retrofitted at launch review; the constraint belongs in the framing block, which is why [Section 1.6](#) put it there.

Remark 1.5 (Data protection). Behavioural mobile-money data is personal data. Kenya’s Data Protection Act (2019) and its cross-border kin (Nigeria’s NDPA, South Africa’s POPIA, GDPR where applicable) impose purpose limitation, minimization, and consent requirements on exactly the feature pipelines this course builds. The engineering consequences — documented feature provenance, deletion workflows, access control on raw logs — are part of the MLOps weeks, not a legal footnote.

1.12 From Model to System: MLOps and the Agentic Outlook

1.12.1 The MLOps spine of this course

Suppose Weeks 2–7 succeed and you hold a validated model beating the scorecard. [Figure 1.3](#) has already warned you: you are perhaps a quarter of the way to production. The second half of this unit — the half that separates a data *scientist* from an ML *engineer* — builds the system around the model, along the spine of [Fig. 1.8](#): a model that is **served** behind a typed API (FastAPI), **containerized** for reproducible deployment (Docker), **tracked** with versioned experiments, parameters, and artifacts (MLflow), **tested and shipped** through continuous integration (GitHub Actions), and **observed** in production with input-drift and performance monitoring (Evidently) — whose alerts close the loop of [Fig. 1.4](#) by triggering retraining.

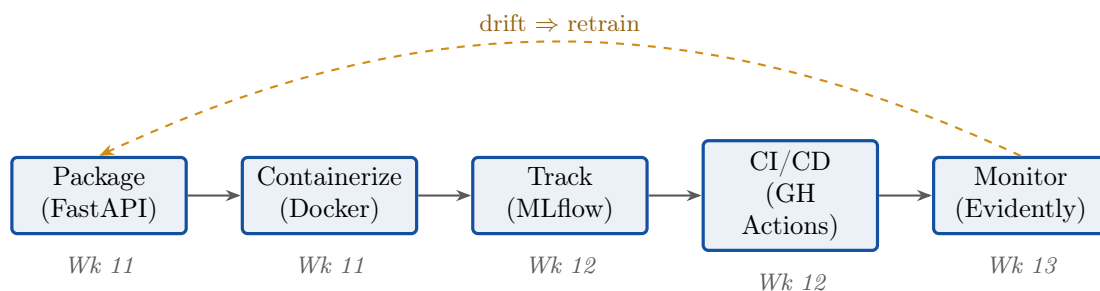


Figure 1.8: The MLOps spine of the course, with the week in which each stage is built for the running credit-scoring system. The dashed amber edge is the same monitoring-to-retraining loop as in [Fig. 1.4](#), now with concrete tooling.

A preview of how little ceremony the serving layer needs — and how directly the leakage-safe pipeline of [Listing 1.5](#) becomes the production artifact:

```

1 # app.py -- minimal scoring service (Week 11 builds this out)
2 import joblib
3 import pandas as pd
4 from fastapi import FastAPI
5 from pydantic import BaseModel
6
7 app = FastAPI(title="thin-file-scorer")
8 model = joblib.load("artifacts/credit_pipeline_v1.joblib")

```

```

9
10 class Application(BaseModel):
11     txn_count_90d: int
12     inflow_median_90d: float
13     balance_vol_90d: float
14     agent_diversity_90d: int
15     airtime_spend_30d: float
16     region: str
17     business_sector: str
18     wallet_tier: str
19
20 @app.post("/score")
21 def score(a: Application):
22     x = pd.DataFrame([a.model_dump()])
23     p = float(model.predict_proba(x)[: , 1])
24     return {"p_default": p, "model_version": "v1"}

```

Listing 1.7: Serving preview (built properly in Week 11): the *same* serialized pipeline object used in training scores live requests, eliminating training/serving skew by construction.

1.12.2 Where LLMs and agents fit — and where they do not

No honest 2026 treatment of applied ML can ignore large language models and agentic systems; no rigorous one can pretend they replace the classical core. The mature position, which this course defends and Week 10 implements, is *hybrid*:

Genuinely useful at the edges of the system.

- **Retrieval-augmented generation (RAG)** over policy manuals, KYC regulation, and product documentation, giving underwriters and support staff grounded, citable answers instead of hallucinated ones.
- **LLM feature extraction** from unstructured inputs — scanned statements, free-text loan purposes, customer chat — converting language into structured features that the classical scorer consumes.
- **Agentic orchestration:** an agent that gathers an applicant’s data, calls the scoring API of [Listing 1.7](#), drafts a reason-coded explanation, and *routes borderline cases to a human* — automation of the workflow around the decision, with the human kept in the loop where stakes demand it.

Still the wrong tool for the core decision.

- **Core credit scoring on tabular data.** Gradient-boosted tree ensembles (Week 4) still dominate LLMs on tabular problems in accuracy, latency, cost, *and* auditability; an LLM asked to emit a default probability is an uncalibrated, unauditable detour.
- **Anything requiring calibrated probabilities and regulatory explainability.** [Equation \(1.6\)](#) is only optimal for calibrated $\hat{p}$; regulators require reason codes tied to model inputs. Neither survives a stochastic text generator at the core.
- **Hard latency and cost budgets at the edge.** A sub-second, sub-cent scoring call over a congested mobile network is served by a compact classical model, not a billion-parameter decoder.

[Figure 1.9](#) sketches the resulting reference architecture for the running case — the shape the capstone system is expected to take.

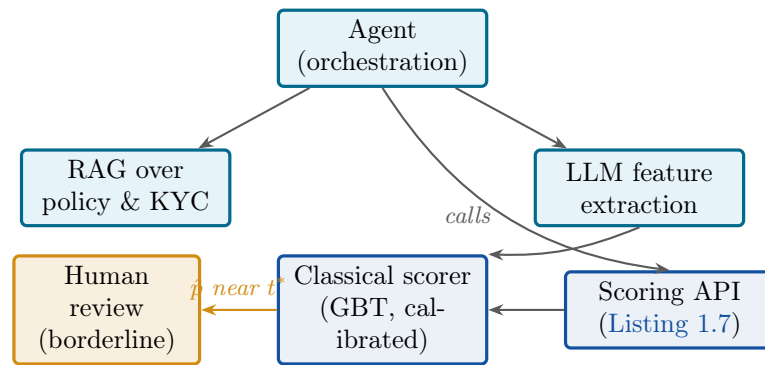


Figure 1.9: The hybrid reference architecture: LLM components (cyan) handle language, retrieval, and orchestration; the calibrated classical core (blue) makes the credit decision behind a versioned API; borderline scores near the cost-optimal threshold t^* of Eq. (1.6) route to a human (amber).

The architectural summary: **classical models do the deciding; LLMs and agents handle language, retrieval, and orchestration around them.** Keep this hybrid picture in view during the deep-learning weeks — it is the design stance the capstone rubric rewards.

1.13 Chapter Summary

1. ML *induces* rules from data (Fig. 1.1); Mitchell’s $\langle T, E, P \rangle$ definition forces the task, experience, and metric to be named before modelling begins.
2. Supervised learning is empirical risk minimization (Eq. (1.2)); the entire engineering apparatus of applied ML exists to close the gap between $\hat{R}$ and R — *and keep it closed* after deployment.
3. The taxonomy (supervised / unsupervised / self-supervised / RL) and the operational axes (batch vs. online, instance- vs. model-based) determine your data requirements, evaluation design, and infrastructure cost.
4. The model is the small green box (Fig. 1.3); the system around it — data, serving, monitoring, configuration — is the real work, and its failure modes (skew, drift, leakage, irreproducibility) are amplified in thin-file, alternative-data, edge-latency African deployments.
5. The lifecycle is a monitoring-driven *loop* (Fig. 1.4), not a waterfall; 80% of the work lives in framing and data.
6. Framing, metrics, splits, and leakage decide success before any model is trained: optimize expected cost (Eq. (1.5)), not accuracy; split by time and group; audit every feature for leakage (Definition 1.3).
7. Default to a strong, leak-free baseline and report lift over it; the bias–variance decomposition (Eq. (1.7)) explains why added capacity is not free.
8. Fairness is a choice among conflicting, testable criteria; proxy features defeat “fairness through unawareness”; per-segment metrics and reason codes are deployment requirements, not afterthoughts (Section 1.11).
9. Modern applied ML is *hybrid*: classical cores for calibrated, auditable decisions; LLMs and agents for language, retrieval, and orchestration at the edges.

Lab 1: Development Environment and a Baseline Pipeline (1.5 h)

Today’s practical sets up the professional toolchain and walks the outer ring of the lifecycle once, in miniature.

1. **Environment.** Install and configure Git and VS Code; create an isolated environment (`venv` or `conda`); pin dependencies in `requirements.txt`. Reproducibility starts here.
2. **Project scaffold.** Lay out the repository structure used all semester: `src/`, `data/` (git-ignored), `notebooks/`, `tests/`, `README.md`.
3. **Data.** Load the Week 1 transactional dataset; profile missingness, class imbalance, and the time span of the records.
4. **Honest split.** Implement a temporal train/validation/test split as in [Fig. 1.6](#); verify class ratios per block.
5. **Baseline pipeline.** Build the end-to-end pipeline of [Listing 1.5](#): preprocess → fit → evaluate, reporting PR–AUC and a precision–recall curve against the majority-class floor of [Listing 1.6](#).
6. **Version control.** Commit with a meaningful history and open a pull request for peer review.

Deliverable: notebook plus repository link on the Virtual Campus, with a three-line framing block (objective, metric, baseline) at the top — the same block that will head every deliverable in this course.

Assignment 1: Framing and Baseline Report (graded, due Week 2)

Learning objectives. Demonstrate the Week-1 discipline end to end: (i) frame a business problem as an ML task with a defensible metric and baseline; (ii) design honest, temporally sound splits; (iii) build a leakage-free baseline pipeline; (iv) report results as lift over baseline with per-segment breakdowns.

Dataset requirements. The Week-1 transactional credit dataset distributed on the Virtual Campus (or, with prior approval, a comparable real-world tabular dataset with a timestamp, a rare binary outcome, and at least one repeated-entity key — no toy datasets). Document row counts, time span, base rate, and missingness.

Task. Submit a repository containing: (1) a framing block — objective, ML task, metric with cost rationale, incumbent baseline, constraints; (2) an EDA notebook answering the four questions of [Section 1.7](#); (3) a temporal train/validation/test split with a stated group key; (4) the baseline pipeline of [Listing 1.5](#) evaluated against the majority-class floor; (5) a one-page decision memo in the structure of [Section 1.10](#), including one paragraph on the leakage audit of your features.

Grading rubric (20 marks).

Criterion	Marks	What earns full marks
Framing block	4	metric tied to error costs; baseline stated quantitatively
EDA depth & honesty	4	all four questions of Section 1.7 answered with evidence; training window only
Split design	4	temporal + stratified + group-aware, verified empirically
Baseline pipeline	4	all fitted steps inside the pipeline; runs end-to-end from a clean clone
Decision memo & leakage audit	4	business-unit headline; explicit per-feature as-of reasoning

Penalty clauses: any leakage found by the graders caps the pipeline criterion at 1 mark; an irreproducible repository (missing pins, absolute paths, manual steps) caps the total at 12.

Exercises

- (Framing.)** Write the full $\langle T, E, P \rangle$ specification ([Definition 1.1](#)) and the four-part framing block of [Section 1.6](#) for: (a) streaming fraud detection on a mobile-money rail with a 300-alerts/day operations team; (b) predicting 30-day readmission from clinical pathway data. State the baseline each system must beat.
- (ML or not.)** For each of the following, argue in at most five sentences whether ML is justified over a rule, using [Table 1.1](#): (a) blocking transactions above a regulatory limit; (b) detecting SIM-swap account takeover; (c) routing support tickets by language.
- (ERM.)** Show that the constant predictor minimizing empirical squared error is the sample mean, and the one minimizing empirical absolute error is the sample median. Relate the result to the RMSE/MAE choice in [Section 1.6.2](#).
- (Cost-sensitive thresholds.)** Derive [Eq. \(1.6\)](#) from first principles by comparing the conditional expected cost of approving versus declining an applicant with default probability $\hat{p}$. Then extend the derivation to a three-way policy (approve / refer-to-human / decline) with a per-referral review cost c_R , and characterize the two thresholds.
- (Leakage audit.)** Classify each proposed feature for the origination-time credit model by leakage species ([Section 1.8.2](#)), or declare it safe, with justification: (a) number of repayment reminders sent; (b) median mobile-money balance over the 90 days *before* application; (c) standard-deviation of balance over the loan term; (d) a merchant-ID one-hot encoding, with random row-wise splits.
- (Bias–variance.)** Complete the omitted algebra in [Eq. \(1.7\)](#): show that both cross-terms vanish, stating exactly where independence of ε and S , and the definition of $\bar{f}$, are used.
- (Splits, empirically.)** Using [Listing 1.5](#) as a skeleton on any time-stamped tabular dataset, compare validation PR–AUC under (a) a random shuffled split and (b) a temporal split. Explain the direction of the gap and state which number an honest report may cite.
- (Fairness.)** Two regions have default base rates of 3% and 9%. Show that a single calibrated model with one global threshold cannot satisfy demographic parity, and discuss

— in the language of [Section 1.11](#) — which criterion you would propose to the regulator for the running case, and what you would monitor per segment after deployment.

9. (**Hybrid architecture.**) Sketch (one diagram, one page) the architecture of an agentic underwriting assistant that uses RAG over lending policy, calls the scoring API of [Listing 1.7](#), and escalates borderline cases to a human. Mark which components are classical ML, which are LLM-based, and where calibrated probabilities must be preserved end-to-end.

Key Terms

Term	Working definition (chapter reference)
Empirical risk minimization	choosing f to minimize average training loss plus regularization (Eq. (1.2))
Generalization gap	$R(\hat{f}) - \hat{R}(\hat{f})$; the quantity all evaluation discipline protects (Eq. (1.3))
Bayes risk	the irreducible error floor of the problem itself (Section 1.2.5)
Data / concept drift	change in $P(x)$ vs. change in $P(y x)$ after deployment (Definition 1.2)
Training/serving skew	offline and online feature computations diverging (Section 1.4)
Leakage	training on information unavailable at prediction time (Definition 1.3)
Selective labels	outcomes observed only for approved cases, biasing all future training data (Section 1.4)
Calibration	scores that match observed frequencies; precondition for Eq. (1.6)
PR–AUC	area under the precision–recall curve; preferred ranking summary under imbalance (Fig. 1.5)
Baseline lift	improvement over the simplest defensible predictor; the only honest headline (Section 1.9.2)
Shadow mode / champion–challenger	low-risk promotion patterns for a new model (Section 1.10)
Equalized odds	equal per-group TPR and FPR; one of several conflicting fairness criteria (Section 1.11)

Further Reading

[Géron \(2022, Ch. 1–2\)](#) covers the landscape and the end-to-end project walkthrough that today’s lab mirrors. [Huyen \(2022, Ch. 1–2\)](#) develops the systems view — batch vs. streaming, framing, and the business–ML objective gap — in production depth. [Sculley et al. \(2015\)](#) is the short, essential paper behind [Fig. 1.3](#); read it before Week 11. [Kaufman et al. \(2012\)](#) formalizes leakage with case studies. [Bishop and Bishop \(2024, Ch. 1\)](#) gives a complementary, probability-first account of the learning problem, and [Breiman \(2001\)](#) remains the sharpest essay on the culture gap between data modelling and algorithmic prediction. On the African fintech side, [Bjorkegren and Grissen \(2020\)](#) is the canonical evidence that behavioural mobile data predicts repayment.

Part II

Deep Learning and Generative AI

Part III

MLOps and Production Systems

Appendices

Solutions to Selected Exercises

This appendix collects worked solutions to the end-of-chapter exercises. Attempt each exercise seriously before reading its solution: in this course, as in production, the diagnostic instinct is built by being wrong first in private.

Chapter 1: Introduction to ML and End-to-End ML Projects

Solution 1 (Framing)

(a) Streaming fraud detection with a 300-alerts/day operations team.

Mitchell specification.

- T : score each transaction at initiation, before settlement, and emit an alert for those most likely to be fraudulent.
- E : historical transaction logs with fraud labels arriving with a delay of days to weeks, from chargebacks, customer complaints, and investigator dispositions; labels are incomplete (undetected fraud is labelled “legitimate”).
- P : fraud value captured (or recall) *at a fixed budget of 300 alerts per day* — operationally, precision@ k with $k = 300/\text{day}$, weighted by transaction value.

Framing block. **Objective:** minimize net fraud losses subject to the investigation-capacity constraint and a false-positive customer-friction budget. **ML task:** binary classification (or anomaly scoring) on a *stream*; the drift speed of adversarial behaviour argues for online or frequently retrained batch learning (Table 1.4), evaluated prequentially. **Metric:** value-weighted recall at the 300-alert budget; secondary, time-to-detect for new fraud patterns. **Baseline:** the incumbent rules engine — its capture rate at the same alert volume is the number to beat. **Constraints:** per-transaction scoring latency in the tens of milliseconds; adversarial drift; delayed and censored labels; rollback capability for bad online updates.

Two honesty points earn full marks: the label censoring (undetected fraud contaminates the negatives, so measured recall is optimistic), and the feedback loop — blocking transactions changes which frauds are ever observed, the streaming analogue of the selective-labels problem of Section 1.4.

(b) Thirty-day readmission from clinical pathway data.

Mitchell specification.

- T : at discharge, estimate the probability that the patient is readmitted within 30 days.
- E : historical admission episodes — diagnoses, procedures, length of stay, vitals trajectories, prior utilization — joined to the observed readmission outcome.

- *P*: recall (or expected cost captured) at the capacity of the post-discharge intervention programme (follow-up calls, home visits); PR-AUC as the threshold-free summary, given a low base rate.

Framing block. **Objective:** allocate a fixed post-discharge intervention capacity to the patients whose readmission it is most likely to prevent. **ML task:** binary classification at the discharge-decision point; batch retraining is adequate (clinical drift is slow relative to fraud). **Metric:** recall at the intervention budget; per-segment calibration, since resource allocation across wards and demographics will be scrutinized. **Baseline:** the incumbent clinical risk score (e.g. a LACE-style index) or current clinician triage. **Constraints:** explainability to clinicians; data-protection compliance for health records; fairness across demographic groups; label subtleties — death within 30 days is a competing risk that must be handled explicitly, not silently coded as “no readmission”.

The deepest framing observation: the deployment intent is *interventional* (we act on high-risk patients), so post-deployment outcome data is shaped by the model’s own decisions — future retraining must account for the intervention, exactly the hidden feedback loop of [Section 1.4](#).

Solution 2 (ML or not)

(a) **Blocking transactions above a regulatory limit: rule.** The requirement *is* a rule — crisp, complete, auditable, and legally fixed. ML could only approximate it, adding error where zero error is mandated and destroying auditability. Right-hand column of [Table 1.1](#) on every criterion.

(b) **SIM-swap account takeover: ML, wrapped in rules.** The signal is high-dimensional and behavioural (device changes, dormancy-then-drain patterns, atypical beneficiaries), and the adversary adapts, so any static rulebook rots within months — the left-hand column on every criterion. The mature design keeps hard rules as guardrails (e.g. mandatory step-up authentication within N hours of a SIM change) with an ML risk score ranking the grey zone; the rules also generate the labelled outcomes that train the model, the “rules first, ML second” pattern of [Section 1.2.3](#).

(c) **Routing support tickets by language: ML, but off the shelf.** Hand-written rules (character ranges, keyword lists) are brittle across code-switching and short texts, so learning is justified; but language identification is a solved commodity problem, and a lightweight pretrained classifier meets the accuracy, latency, and cost bar. The build-or-not lesson: “use ML” does not imply “train your own” — the total cost-of-ownership argument of [Table 1.1](#) favours the smallest maintained artifact that clears the requirement.

Solution 3 (ERM: mean and median as optimal constants)

Squared error. For a constant predictor c , the empirical risk is

$$J(c) = \frac{1}{n} \sum_{i=1}^n (y_i - c)^2, \quad J'(c) = -\frac{2}{n} \sum_{i=1}^n (y_i - c),$$

and setting $J'(c) = 0$ gives $c^* = \frac{1}{n} \sum_i y_i = \bar{y}$, the sample mean; $J''(c) = 2 > 0$ confirms the minimum.

Absolute error. $G(c) = \frac{1}{n} \sum_i |y_i - c|$ is convex and piecewise linear, so we argue by subgradient. For c not equal to any y_i ,

$$G'(c) = \frac{1}{n} \left(\#\{i : y_i < c\} - \#\{i : y_i > c\} \right),$$

which is negative while fewer than half the points lie below c and positive once more than half do. The minimum is therefore attained where the two counts balance: at the sample median

(for even n , anywhere on the interval between the two middle order statistics — the minimizer need not be unique).

Connection to Section 1.6.2. Minimizing RMSE trains the model toward the conditional *mean*, which is pulled by extreme outcomes; minimizing MAE trains it toward the conditional *median*, which is robust to them. Choosing between Eq. (1.4) is therefore choosing *which summary of the target distribution the model learns* — a business decision (how do errors actually cost?) dressed as a metric choice.

Solution 4 (Cost-sensitive thresholds)

Two-way derivation. For an applicant with calibrated default probability $\hat{p}$, compare conditional expected costs:

$$\mathbb{E}[C \mid \text{approve}] = \hat{p} c_{\text{FN}}, \quad \mathbb{E}[C \mid \text{decline}] = (1 - \hat{p}) c_{\text{FP}}.$$

Decline exactly when $\hat{p} c_{\text{FN}} > (1 - \hat{p}) c_{\text{FP}}$; rearranging,

$$\hat{p} > \frac{c_{\text{FP}}}{c_{\text{FP}} + c_{\text{FN}}} = t^*,$$

which is Eq. (1.6). The derivation *uses* calibration: $\hat{p}$ must be interpretable as a probability for the expectations to be meaningful, which is why calibration is a precondition, not a nicety.

Three-way extension. Add the action *refer*, costing a review fee c_R and (idealizing) resulting in the correct decision, so it incurs no error cost. The three conditional expected costs are $\hat{p} c_{\text{FN}}$ (approve), $(1 - \hat{p}) c_{\text{FP}}$ (decline), and c_R (refer). Comparing pairwise:

- approve beats refer iff $\hat{p} c_{\text{FN}} < c_R$, i.e. $\hat{p} < t_L := c_R / c_{\text{FN}}$;
- decline beats refer iff $(1 - \hat{p}) c_{\text{FP}} < c_R$, i.e. $\hat{p} > t_U := 1 - c_R / c_{\text{FP}}$.

The optimal policy is therefore: *approve* if $\hat{p} < t_L$, *refer* if $t_L \leq \hat{p} \leq t_U$, *decline* if $\hat{p} > t_U$. The referral band is non-empty iff $t_L < t_U$, i.e.

$$c_R < \frac{c_{\text{FN}} c_{\text{FP}}}{c_{\text{FN}} + c_{\text{FP}}},$$

the harmonic combination of the two error costs; cheap review widens the band, expensive review collapses it to the single threshold t^* — which, one checks, always lies inside $[t_L, t_U]$ when the band exists. This is exactly the “ $\hat{p}$ near t^* routes to a human” edge of Fig. 1.9, now with the band derived rather than asserted. (A fuller model would let the human err too; the band then narrows in proportion to reviewer accuracy, a good extension discussion.)

Solution 5 (Leakage audit)

(a) **Number of repayment reminders sent: target leakage.** Reminders exist only after origination and are triggered by delinquency — the feature is a downstream *consequence* of the label. At application time it is identically empty, so a model trained with it learns to read the outcome off its own paperwork and collapses in production (Definition 1.3).

(b) **Median mobile-money balance over the 90 days before application: safe** — *conditional* on discipline. The window $[t_0 - 90\text{d}, t_0]$ respects the as-of contract of Table 1.6. The residual risks to state for full marks: the feature must be computed from event data as it stood at t_0 (late-arriving corrections or restated ledgers re-introduce temporal leakage), and the training-time computation must share code with the serving-time computation or training/serving skew replaces leakage as the failure mode.

(c) **Standard deviation of balance over the loan term: temporal leakage.** The loan term lies strictly *after* the origination decision the model is meant to inform. The feature is

computable in the historical warehouse (which is why it slips into training sets) but can never be computed at prediction time. Textbook violation of the as-of rule.

(d) Merchant-ID one-hot with random row-wise splits: group leakage. Repeat merchants appear on both sides of a random split, so the identifier lets the model memorize *who* defaulted rather than learn *which behaviours* predict default; validation scores inflate and the model is useless for the new applicants it exists to score (whose IDs it has never seen). Remedies: split by merchant group *and* time; drop the raw identifier in favour of behavioural aggregates with proper as-of semantics; if entity-level history is genuinely predictive, encode it as backdated features (e.g. prior loans, prior repayment record as of t_0), which are legitimate.

Solution 6 (Bias–variance: the vanishing cross-terms)

Setup as in [Section 1.9.1](#): $y = g(x) + \varepsilon$ with $\mathbb{E}[\varepsilon] = 0$, $\text{Var}(\varepsilon) = \sigma^2$, ε independent of the training sample S ; $\hat{f}$ is trained on S ; $\bar{f}(x) = \mathbb{E}_S[\hat{f}(x)]$. Fix x and expand (all expectations over S and ε jointly):

First cross-term. Expanding $\mathbb{E}[(g + \varepsilon - \hat{f})^2] = \mathbb{E}[(g - \hat{f})^2] + 2\mathbb{E}[\varepsilon(g - \hat{f})] + \mathbb{E}[\varepsilon^2]$, the middle term factorizes because ε is independent of S and $\hat{f}$ is a function of S (and $g(x)$ is a constant for fixed x):

$$\mathbb{E}[\varepsilon(g - \hat{f})] = \mathbb{E}[\varepsilon] \mathbb{E}_S[g - \hat{f}] = 0 \cdot \mathbb{E}_S[g - \hat{f}] = 0.$$

Independence is used to factorize; the zero mean of ε kills the product. This leaves $\mathbb{E}[(g - \hat{f})^2] + \sigma^2$.

Second cross-term. Insert and subtract $\bar{f}$: $(g - \hat{f}) = (g - \bar{f}) + (\bar{f} - \hat{f})$, so

$$\mathbb{E}_S[(g - \hat{f})^2] = (g - \bar{f})^2 + 2(g - \bar{f}) \mathbb{E}_S[\bar{f} - \hat{f}] + \mathbb{E}_S[(\bar{f} - \hat{f})^2].$$

Here $(g - \bar{f})$ is a deterministic quantity (both terms are fixed numbers once x is fixed), so it exits the expectation; and by the *definition* of $\bar{f}$, $\mathbb{E}_S[\bar{f} - \hat{f}] = \bar{f} - \mathbb{E}_S[\hat{f}] = 0$. The cross-term vanishes, leaving $\text{Bias}^2 + \text{Variance} + \sigma^2$, which is [Eq. \(1.7\)](#). Summary of where each assumption bites: independence of ε and S kills the first cross-term; the definition of $\bar{f}$ as the S -average kills the second.

Solution 7 (Splits, empirically)

A model answer on synthetic data with controlled drift (students' numbers on real data will differ in size, not in sign):

```

1 import numpy as np
2 import pandas as pd
3 from sklearn.linear_model import LogisticRegression
4 from sklearn.metrics import average_precision_score
5 from sklearn.model_selection import train_test_split
6 from sklearn.pipeline import Pipeline
7 from sklearn.preprocessing import StandardScaler
8
9 rng = np.random.default_rng(8401)
10 n, d = 30_000, 10
11 t = np.sort(rng.uniform(0, 1, n))          # "application date" in [0,1]
12 X = rng.normal(size=(n, d))
13 w = np.array([2.0, -1.5, 1.0, 0.5, 0, 0, 0, 0, 0, 0])
14 drift = 2.5 * t                            # concept drifts over time
15 logits = X @ w + drift * X[:, 3] - 3.2
16 y = rng.binomial(1, 1 / (1 + np.exp(-logits)))
17
18 model = Pipeline([("s", StandardScaler()),
19                  ("m", LogisticRegression(max_iter=2_000,
20                  class_weight="balanced"))])

```

```

21
22 # (a) Random shuffled split -- time ignored
23 Xa_tr, Xa_te, ya_tr, ya_te = train_test_split(
24     X, y, test_size=0.25, stratify=y, random_state=8401)
25 pa = model.fit(Xa_tr, ya_tr).predict_proba(Xa_te)[: , 1]
26
27 # (b) Temporal split -- train on the past, test on the future
28 cut = int(0.75 * n) # rows already time-ordered
29 pb = (model.fit(X[:cut], y[:cut])
30       .predict_proba(X[cut:]))[: , 1])
31
32 print(f"random split PR-AUC: {average_precision_score(ya_te, pa):.3f}")
33 print(f"temporal split PR-AUC: {average_precision_score(y[cut:], pb):.3f}")

```

Listing B.1: Random vs. temporal split under drift: the random split reports an optimistic number that no deployment can honour.

Direction and explanation. The random split scores higher. Under shuffling, training and test rows are interleaved in time, so the training set contains examples from the *same regime* as every test example — the model has effectively seen the future, and the drift term never has to be extrapolated. The temporal split forces genuine forward prediction into a drifted regime, which is exactly what deployment does (Fig. 1.6). **The honest number is the temporal one;** the gap between the two is not noise but a first, free estimate of how much drift will cost in production — worth reporting in the decision memo of Section 1.10.

Solution 8 (Fairness)

Let F_A, F_B be the score CDFs of the calibrated model $\hat{p}$ in the two regions, with base rates $\pi_A = \mathbb{E}[\hat{p} \mid A] = 0.03$ and $\pi_B = \mathbb{E}[\hat{p} \mid B] = 0.09$ (calibration plus the tower property force the group mean score to equal the group base rate). Approval is $\hat{p} \leq t$ for a single global t .

Impossibility across thresholds. Demographic parity at *every* threshold would mean $F_A(t) = F_B(t)$ for all t , i.e. identical score distributions — hence identical means, contradicting $0.03 \neq 0.09$. So there is no global-threshold policy family satisfying parity uniformly.

The single-threshold case. At one fixed t , parity requires the accident $F_A(t) = F_B(t)$ at that exact point. But any calibrated model that actually uses the risk information distinguishing the regions places more probability mass at higher scores for region B (in the natural case, F_B is stochastically dominated by — i.e. sits below — F_A), giving $F_B(t) < F_A(t)$: region B’s approval rate is strictly lower at every useful threshold. Even in contrived cases where the CDFs cross at some t , the coincidence is knife-edge — it fails for neighbouring thresholds and evaporates at the first retrain — so it cannot ground a policy. The structural conclusion: with unequal base rates, a calibrated score plus a single global threshold *cannot* deliver demographic parity; achieving parity would require group-specific thresholds, which raises its own legal and ethical questions, or abandoning calibration — the conflict promised in Section 1.11.

Proposal to the regulator. A defensible position for the running case: retain per-group *calibration* (an uncalibrated score corrupts every downstream decision through Eq. (1.6)) and adopt *equal opportunity* — equal true-positive rates among repayers — as the operative criterion, i.e. good borrowers in both regions face the same approval odds; then justify any residual approval-rate gap by the base-rate difference while committing to close it upstream (product design, data enrichment for the thin-file region). **Post-deployment monitoring:** per-region approval rate, TPR/FPR among matured labels, calibration curves, and their drift over time — the fairness axis of the Week-13 monitors, reported segment-by-segment as Section 1.11 prescribes.

Solution 9 (Hybrid architecture)

A full-marks answer reproduces the topology of Fig. 1.9 with the component classification made explicit:

- **Classical ML (the decision core).** The gradient-boosted or logistic scoring model with its leakage-safe feature pipeline (Listing 1.5), served behind the versioned API of Listing 1.7. It alone emits $\hat{p}$; it is trained, calibrated, validated, and monitored under the discipline of Chapters 1–4 and 11–13.
- **LLM components (the language edges).** (i) A RAG service over lending policy, KYC regulation, and product documents, answering underwriter questions with citations; (ii) an extraction model converting unstructured inputs (statements, free-text loan purpose, chat) into *typed features* that enter the classical pipeline — validated like any other upstream data source; (iii) an agent orchestrating the workflow: gather data, call the scoring API, draft the reason-coded communication.
- **Human review.** Applications with $\hat{p} \in [t_L, t_U]$ — the referral band derived in Solution 4 — route to a credit officer, with the agent’s brief attached.

Where calibration must be preserved end-to-end — the marking crux: the probability $\hat{p}$ must flow *numerically unaltered* from the scorer through the API to the thresholding policy and into any human-facing brief. The agent may *explain* the score; it must never *re-estimate*, round, re-rank, or paraphrase it into a different number, and no LLM output may feed back into the decision path — the moment a stochastic text generator sits between the calibrated score and the threshold of Eq. (1.6), the optimality of the policy, the audit trail, and the fairness monitoring of Solution 8 are all void. Diagrammatically: every arrow into the approve/refer/decline node originates at the classical scorer; LLM arrows terminate at humans or at the scorer’s *inputs*, never at its output.

B.1 Chapter 2: Data Preparation and Feature Engineering

Solution to Exercise 2.1. Consider the electronic medical records of a network of Kenyan outpatient clinics.

MCAR. A batch of laboratory results is lost when the connection between the lab analyser and the records system fails for an afternoon. Which patients’ rows are affected is determined by the timing of the outage, not by the patients or their values: $P(M | X) = P(M)$ (Definition 2.1). Mean imputation of the missing lab values yields an *unbiased* estimate of the population mean (the missing rows are a random sample), although it understates the variance (Exercise 2.3).

MAR. Blood-pressure readings are missing far more often at rural dispensaries that lack a working monitor than at urban clinics. Conditional on the *observed* facility-type column, missingness carries no further information about the reading itself (Definition 2.2). Unconditional mean imputation is *biased* whenever blood pressure differs systematically by facility type (urban and rural patient populations differ): the imputed mean is dominated by urban readings. Imputation that conditions on facility type — a group-wise mean, or the model-based imputers of the chapter — removes the bias.

MNAR. HIV-test results are missing because some patients decline testing, and the propensity to decline is associated with the patient’s own (unobserved) status and risk perception: $P(M | X)$ depends on X_{mis} (Definition 2.3). Mean imputation is *biased*, and no imputation scheme built from the observed data alone removes the bias; per Remark 2.1, the honest treatment records the refusal itself as an indicator feature and interprets downstream estimates with the selection mechanism in mind.

In summary, mean imputation is unbiased for the mean only under MCAR; under MAR it requires conditioning on the drivers of missingness; under MNAR it is biased regardless.

Solution to Exercise 2.2. Let $R_i \in \{0, 1\}$ indicate that row i is a complete case, and let X_i be its (complete-data) value. MCAR states $P(R | X) = P(R)$, i.e. $R \perp X$. The density of X among complete cases is, by Bayes' rule,

$$f(x | R = 1) = \frac{P(R = 1 | X = x) f(x)}{P(R = 1)} = \frac{P(R = 1) f(x)}{P(R = 1)} = f(x),$$

where the middle equality uses independence. The complete cases are therefore distributed identically to the full population — a random subsample. Consequently, with $n_{cc} = \sum_i R_i$ the (random) number of complete cases,

$$\mathbb{E} \left[\frac{1}{n_{cc}} \sum_{i: R_i=1} X_i \middle| n_{cc} > 0 \right] = \mathbb{E}[X],$$

by taking the expectation first conditionally on the pattern $(R_1, \dots, R_n)$ — under which each retained X_i has mean $\mathbb{E}[X]$ by independence — and then over the pattern. The complete-case mean is unbiased; the only cost of MCAR missingness is the loss of efficiency from the reduced sample size. $\square$

Solution to Exercise 2.3. With point-estimate imputation (`sample_posterior=False`), every imputed entry is set to the conditional-mean prediction $\hat{\mathbb{E}}[X_{\text{mis}} | X_{\text{obs}}]$ of the final regression sweep of Eq. (2.2). Decompose the true variance by the law of total variance:

$$\text{Var}(X) = \underbrace{\mathbb{E}[\text{Var}(X | X_{\text{obs}})]}_{\text{residual spread}} + \underbrace{\text{Var}(\mathbb{E}[X | X_{\text{obs}}])}_{\text{explained spread}}.$$

The imputed values realise only the second term: they sit exactly on the conditional-mean surface, with zero residual spread. The completed column's sample variance therefore converges to $\text{Var}(\mathbb{E}[X | X_{\text{obs}}])$, understating $\text{Var}(X)$ by the entire residual component $\mathbb{E}[\text{Var}(X | X_{\text{obs}})]$, and the understatement grows with the missing fraction.

Downstream, any procedure that treats the completed data as if observed — ordinary least squares, a t -test, a confidence interval for a feature mean — inherits standard errors that are too small: intervals are too narrow and p -values anti-conservative, because the analysis pretends the imputed entries carry the same information as real measurements. The remedies are those of proper multiple imputation: draw fills from the posterior (`sample_posterior=True`), create M completed datasets, and combine estimates with Rubin's rules, which add the between-imputation variance the point-estimate shortcut discards.

Solution to Exercise 2.4. Two mechanisms, both consistent with the chapter, explain a *drop* in AUC after trimming $|z| > 3$ rows.

(a) *The outliers were the signal.* In credit risk the extreme tail of behaviour is disproportionately informative about default: erratic whale deposits, balance-inflation spikes, abnormal withdrawal bursts (see the pitfall in Section 2.3). Deleting $|z| > 3$ rows removes a non-random, minority-class-enriched slice of the training data — defaulters exhibit extreme behaviour more often than repayers — so the model loses precisely the discriminative region of the feature space, and its ranking quality on an untrimmed evaluation set falls.

(b) *The analyst created a training-serving mismatch.* Rows were removed from the *training* data, but the validation/test data (and, eventually, production traffic) still contain extreme rows. The model is now evaluated on a region of the input space it was never fitted on — a self-inflicted violation of the distribution-overlap requirement of Section 2.6. A subsidiary

aggravating factor: on heavy-tailed transaction data the Gaussian $|z| > 3$ rule flags a large fraction of perfectly legitimate rows (the z-score's μ and σ are themselves inflated by the tail), so the deletion is both extensive and systematically targeted at high-activity customers.

Solution to Exercise 2.5. From Eq. (2.7), the maximum maps to

$$x'_{\max} = \frac{x_{\max} - x_{\min}}{x_{\max} - x_{\min}} = 1$$

identically — the scaled value of the largest point is 1 whatever its magnitude, which is exactly why a single extreme outlier cannot be “detected” in the scaled data: it is normalised into the boundary.

Let m be the median. Its scaled value is $m' = (m - x_{\min}) / (x_{\max} - x_{\min})$. Replace the maximum by $10 x_{\max}$ (all other order statistics unchanged); the new scaled median is

$$m'' = \frac{m - x_{\min}}{10 x_{\max} - x_{\min}}, \quad \frac{m''}{m'} = \frac{x_{\max} - x_{\min}}{10 x_{\max} - x_{\min}}.$$

For $x_{\min} = 0$ this ratio is exactly $1/10$: the median's scaled value collapses by a factor of ten, and with it the entire bulk of the distribution is crushed toward 0 while the outlier alone occupies the top of the unit interval. This is the fragility that motivates the robust scaler of Eq. (2.8).

Solution to Exercise 2.6. Within level c , model each target as conditionally Bernoulli, $y_i | \theta_c \sim \text{Bernoulli}(\theta_c)$, and place the conjugate prior

$$\theta_c \sim \text{Beta}(\alpha, \beta), \quad \alpha = m \bar{y}, \quad \beta = m(1 - \bar{y}),$$

so that the prior mean is $\alpha / (\alpha + \beta) = \bar{y}$ (the global rate) and the prior strength — the number of pseudo-observations — is $\alpha + \beta = m$. Observing n_c rows of level c with $s_c = \sum_i y_i = n_c \bar{y}_c$ successes, conjugacy gives the posterior $\theta_c | \text{data} \sim \text{Beta}(\alpha + s_c, \beta + n_c - s_c)$, whose mean is

$$\mathbb{E}[\theta_c | \text{data}] = \frac{\alpha + s_c}{\alpha + \beta + n_c} = \frac{m \bar{y} + n_c \bar{y}_c}{m + n_c} = \text{TE}(c),$$

which is Eq. (2.11) exactly. The limits confirm the shrinkage behaviour: as $n_c \rightarrow 0$, $\text{TE}(c) \rightarrow \bar{y}$ (an unseen or rare level is pulled entirely to the prior), and as $n_c \rightarrow \infty$, $\text{TE}(c) \rightarrow \bar{y}_c$ (abundant evidence overwhelms the prior). The smoothing parameter m is thus interpretable as the number of global-prior pseudo-counts each level must “outweigh” before its own history dominates — and setting it too small is precisely what lets a level seen once memorise its own label (Section 2.5.3).

□

Solution to Exercise 2.7. *Mechanism.* Individually, a start date and an end date are weak predictors of churn. Jointly they encode the label almost perfectly at extraction time: an employee who has churned has a populated `contract_end_date` lying before the snapshot date, whereas a current employee has a null, sentinel, or future-dated value. Any model able to combine the two features (or a feature and the implicit snapshot date) reconstructs “has this contract already terminated?” — which *is* the churn label, or a near-deterministic proxy for it. The end date is a *post-outcome field*, written by the HR system as a consequence of the very event being predicted: cause 6 of Section 2.7.2, hiding behind two innocuous-looking columns that pass a univariate correlation screen.

A screen for pairwise leakage. Extend the univariate screen of Section 2.7 to constructed pairs: (i) for every pair of same-typed columns (two dates, two amounts), automatically generate their difference (and, for amounts, their ratio) and rank these derived features by $|\rho(\cdot, y)|$ or mutual information with the target; (ii) as a model-based variant, fit a depth-2 decision tree on each feature pair alone and flag any pair whose standalone AUC approaches the full model's. Either variant catches start/end-date leakage immediately (the difference is tenure-to-termination;

the tiny tree learns “end date < snapshot”). The screen complements, but does not replace, the lineage question — *which system writes this field, and when?* — that identifies post-outcome fields even when their statistical footprint is subtle.

Solution to Exercise 2.8. Assume the hash of Eq. (2.10) assigns each of the K levels independently and uniformly to one of d buckets. Two given levels collide with probability $1/d$, and there are $\binom{K}{2}$ pairs, so by linearity of expectation the expected number of colliding pairs is

$$\mathbb{E}[\text{\#collisions}] = \binom{K}{2} \frac{1}{d} = \frac{K(K-1)}{2d}.$$

For $K = 10^4$ and $d = 256$:

$$\frac{10^4 \times (10^4 - 1)}{2 \times 256} = \frac{99\,990\,000}{512} \approx 1.95 \times 10^5$$

colliding pairs — with an average bucket load of $K/d \approx 39$ levels, so *every* bucket is shared. This is not necessarily fatal (tree models tolerate the induced noise, and the load is spread evenly), but it quantifies the trade-off: at this cardinality, $d = 256$ buys a fixed 256-column footprint at the price of heavy aliasing, and raising d to 2^{12} – 2^{14} reduces the expected pair collisions by one to two orders of magnitude while remaining far cheaper than a 10^4 -column one-hot encoding.

Solution to Exercise 2.9. *Leakage mechanism.* Support-line contact is not exogenous to repayment trouble. Two concrete routes: (i) borrowers who are already struggling call to ask about extensions, penalties, or restructuring *after* the loan being scored has been disbursed — so if the 7-day window is computed relative to label-observation time (or to “now” in the training warehouse) rather than to the scoring timestamp t , the feature ingests post- t information in direct violation of the as-of rule (Eq. (2.1)); (ii) worse, some inbound calls are *returns of collections outreach* — the collections team calls delinquent borrowers, who call back — making the count a laundered copy of the delinquency flag, exactly the post-outcome pattern of `days_since_collections_call` in Listing 2.11 (cause 6 of Section 2.7.2).

Redesign. Keep the legitimate signal (a customer’s baseline propensity to need support may genuinely correlate with risk) by: fixing the window to end strictly before the scoring timestamp, $[t - 7d, t)$, enforced by the feature platform’s point-in-time semantics (Section 2.8); excluding call types causally downstream of the credit process (collections callbacks, repayment-difficulty queries about an *active* loan), which requires the call-category field and its write-time lineage; and validating the rebuilt feature with the correlation screen — its association with the target should fall from “implausible” to “modest” if the leak, and not the baseline behaviour, was carrying the load.

Solution to Exercise 2.10. A five-sentence rebuttal. First, native NaN routing (?) solves only the mechanical problem of fitting a tree through incomplete rows; it does not answer the diagnostic questions — *why* is the value missing, and is the missingness itself predictive? — that Lab 2’s MCAR/MAR/MNAR audit exists to settle. Second, under MNAR the explicit indicator of Remark 2.1 is a first-class, interpretable feature whose effect can be measured, reviewed for fairness, and explained to a credit officer, whereas a learned default branch direction is an implicit, per-split artefact that cannot. Third, the pipeline must remain model-agnostic: the moment we benchmark against a regulatory logistic scorecard (Eq. (2.13)) or a neural baseline (??), models without native handling need the imputation stage anyway, and comparing models across different missing-data treatments confounds the comparison. Fourth, training-serving consistency (Section 2.8) requires that whatever missing-value semantics the model learned be reproduced exactly in the online path — a fitted, serialised imputer inside the Pipeline of Listing 2.12 is a deployable contract, while “the library handles it” is not. Fifth, most of Lab 2’s effort — parsing, validity rules, entity resolution, leakage hunting — is not imputation at all, and no amount of native NaN support dedupes a customer with three SIM cards.

Bibliography

- Bishop, C. M. and Bishop, H. (2024). *Deep Learning: Foundations and Concepts*. Springer.
- Bjorkegren, D. and Grissen, D. (2020). Behavior revealed in mobile phone usage prediction credit repayment. In *The World Bank Economic Review*, volume 34.
- Breiman, L. (2001). Statistical modeling: The two cultures. *Statistical Science*, 16(3):199–231.
- Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow*. O'Reilly Media, 3rd edition.
- Huyen, C. (2022). *Designing Machine Learning Systems*. O'Reilly Media.
- Kaufman, S., Rosset, S., Perlich, C., and Stitelman, O. (2012). Leakage in data mining: Formulation, detection, and avoidance. *ACM TKDD*, 6(4).
- Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., and Dennison, D. (2015). Hidden technical debt in machine learning systems. In *Advances in Neural Information Processing Systems 28*.

